

NGINX JS DOM — Developer Manual

Contents

1. Introduction	4
1.1. What NGINX JS DOM Is	4
1.2. Intended Audience	4
1.3. A Map of the Moving Parts	4
2. Architecture	5
2.1. Lifecycle Overview	5
2.2. Process and Thread Topology	6
2.3. COM Ownership and Mutation Scope	7
3. Quick Start	7
3.1. Hello World Handler	8
3.2. Reading Request Data	8
3.3. Async Fetch and Transform	9
3.4. Live Upstream Rebalancing	10
4. Request Handlers	10
4.1. Installing a Handler	10
4.2. NginxRequest Reference	11
4.2.1. Identity and Network	11
4.2.2. Headers	11
4.2.3. Body	12
4.2.4. Cookies and Variables	12
4.2.5. Location and Server Context	12
4.2.6. Response	12
4.2.7. Async Utilities	13
4.2.8. Logging	13
4.3. Async Handler Lifecycle	13
4.4. Streaming Responses	14
4.5. Subrequest Fan-Out	14
4.6. Header and Body Filters	14
5. Configuration Object Model (COM)	15
5.1. Overview	15
5.2. NginxServer	16
5.3. NginxLocation	17
5.4. NginxUpstream and Peers	17
5.5. Iterating the Full Config Tree	18
6. Dynamic Reconfiguration	18
6.1. Scope 1 — This Worker Only	18

6.2. Scope 2 — All Workers Atomically	19
6.3. Scope 3 — Add / Remove Servers and Locations	19
6.4. Scope 4 — Per-Request Config Snapshot	20
6.5. Hot SSL Certificate Swap	21
7. Pre-Fork Initialization	21
7.1. <code>js_preprocess</code> — Config Injection During Parse	21
7.1.1. Example: Server Blocks from a JSON File	22
7.1.2. Example: Conditional SSL Config	23
7.1.3. Example: Fetching Config from a Network Service	23
7.1.4. Multiple <code>js_preprocess</code> Directives	24
7.2. <code>js_init_http</code> — Post-Parse Server Injection	24
8. Workers (JS Web Workers)	25
8.1. Basic Worker Pattern	25
8.2. Transferring ArrayBuffers (Zero Copy)	26
8.3. Worker <code>std</code> and <code>os</code> Modules	27
9. SharedWorkers	27
9.1. Concept and Use Cases	27
9.2. Port-Based Pattern (Explicit <code>onconnect</code>)	28
9.3. Broadcast Pattern (Global <code>onmessage</code> + <code>clients</code>)	29
9.4. Dynamic SharedWorker Creation	30
9.5. SharedWorker Flow Chart	30
10. SharedArrayBuffers and Atomics	31
10.1. What They Are	31
10.2. Pre-Fork SABs (Visible to All Workers)	31
10.3. Post-Fork SABs (Worker-Local, Passed via Channels)	32
10.4. Atomics.wait / Atomics.notify	32
11. Accept Control and Cross-Worker Coordination	33
11.1. The Problem	34
11.2. Per-Worker: <code>suspendAcceptance</code> / <code>resumeAcceptance</code>	34
11.3. Cross-Worker: <code>suspendAllWorkers</code> / <code>resumeAllWorkers</code>	34
11.4. SharedWorker Broadcast + Accept Control Pattern	35
11.5. Backlog Behaviour	36
12. Snapshots	37
12.1. Basic Snapshot and Rollback	37
12.2. Sub-Object Snapshots	37
12.3. Read / Write Mode Interaction with Snapshots	38
13. Stream (TCP/UDP) Support	38
14. External Config Sources and Full Reload	39
14.1. Pattern A — File at Startup (<code>js_preprocess</code>)	39
14.2. Pattern B — Network at Startup (<code>js_preprocess</code> + <code>std.urlGet</code>)	39
14.3. Pattern C — Live Reload via Handler (Zero Downtime)	40

14.4. Decision Guide	41
15. Third-Party Module Integration	42
15.1. Overview	42
15.2. Step 1 — Identify the Config Structs	42
15.3. Step 2 — Write the COM Bridge File	42
15.4. Step 3 — Register the Bridge in the COM Initialiser	44
15.5. Step 4 — Use from JavaScript	45
15.6. Location-Level Config	45
16. Resource Protection	46
16.1. Worker Memory Limit	46
16.2. Worker Request Timeout	46
16.3. Error Isolation	47
16.4. What JS Cannot Do	47
17. Reference	47
17.1. Directives	47
17.2. <code>nginx</code> Global Object	48
17.3. <code>nginx.http</code> Object	48
17.4. NginxRequest Methods and Properties	49
17.5. Worker / SharedWorker API	49
17.6. SharedArrayBuffer and Atomics Support Matrix	49
17.7. Log Level Constants	50
18. Appendix — Flow Charts	50
18.1. A: Simple Synchronous Handler	50
18.2. B: Async Handler with Worker Offload	50
18.3. C: Cross-Worker COM Mutation (<code>suspendAllWorkers</code>)	51
18.4. D: SharedWorker State Coordination	51
18.5. E: SAB Lock-Free Counter (Pre-Fork)	51
18.6. F: Pre-Fork Config Generation	51
18.7. G: Live Reload from Config Service	52
18.8. H: Complete <code>nginx.*</code> Object Tree	52
18.8.1. NginxServer properties	53
18.8.2. NginxLocation properties	54
18.8.3. NginxUpstream, NginxPeer, NginxRRPeer	56
18.8.4. NginxSocket and NginxHttpListener / NginxStreamListener	57
18.8.5. NginxStreamServer and NginxStreamProxy	57
18.8.6. NginxRequest and NginxStreamSession	58
18.8.7. Worker and SharedWorker	59
18.8.8. NginxSSL and NginxStreamSSL	60
18.9. I: Full Nginx JS Class Reference	60
18.9.1. Class Index	60
18.9.2. Module COM Class Properties	63

1. Introduction

1.1. What NGINX JS DOM Is

NGINX is deliberately static: its configuration is read once at startup (or reload) and compiled into kernel-level data structures. This rigidity is a feature — it makes nginx predictable, fast, and easy to reason about — but it creates friction whenever runtime adaptability is required: upstream weights that need to track backend health scores, virtual hosts generated from a tenant database, SSL certificates that rotate on a schedule, or request routing rules that depend on live feature-flag state.

NGINX JS DOM embeds [QuickJS](#) (a lightweight [ES2020+](#) JavaScript engine) directly into nginx and exposes the entire nginx configuration as a structured JavaScript object tree called the **Configuration Object Model (COM)**. Through the COM, JavaScript can:

- **Read** the full live configuration of every module in every server and location.
- **Write** configuration properties — upstream peer weights, SSL certificates, location directives, gzip settings, and much more — with immediate effect, either for the current worker or atomically for all workers at once.
- **Generate** server and location blocks at parse time, before nginx has even finished reading its configuration file.
- **Handle requests** in JavaScript, using the same event-driven async model as nginx itself.
- **Offload** CPU-intensive work to background threads (Workers) without blocking the nginx event loop.
- **Maintain shared mutable state** across all nginx worker processes using SharedWorkers and SharedArrayBuffers.

The result is an nginx that can be programmed rather than merely configured — without restarting, without patching C code, and without sacrificing the performance or stability of the core server.

1.2. Intended Audience

This manual assumes:

- Familiarity with nginx configuration syntax and core concepts (workers, upstreams, locations, modules). See the [NGINX Beginner's Guide](#) and the [full module reference](#) if you need a refresher.
- Working knowledge of modern JavaScript (ES2020): [async/await](#), Promises, [import/export](#), [SharedArrayBuffer](#), [Atomics](#). See the [ECMAScript 2020 specification](#) or any ES2020 reference.

You do **not** need to know C or QuickJS internals to use the JS DOM API.

1.3. A Map of the Moving Parts

The sections that follow cover these components:

Component	What it gives you
<code>js_preprocess</code>	Runs JavaScript during config parsing; the script writes config text back into the parser. Use it to generate server/location blocks from files, environment variables, or external sources.
<code>js_init_http</code>	Runs JavaScript after the <code>http {}</code> block is parsed but before directives are merged. Can read already-parsed servers and inject new ones.
<code>js_source</code>	Evaluates JavaScript in the master process after all config has been parsed. This is where you install request handlers, set up Workers and SharedWorkers, and perform any pre-fork initialisation.
Request handlers	JavaScript <code>async</code> functions assigned to <code>location.handler</code> . Each request runs through the nginx event loop; <code>await</code> suspends the handler without blocking any other request.
COM	The <code>nginx</code> global object tree. Read live config values; write mutations that take effect immediately.
Workers	Standard Web Worker API. A per-nginx-worker background thread for CPU-bound work. Does not block the event loop.
SharedWorkers	A single long-lived thread in the master process, connected to every nginx worker via UNIX sockets. The canonical place for mutable state that must be consistent across all workers.
SharedArrayBuffers + Atomics	Binary shared-memory regions (MDN reference) accessible from any thread with zero copy. Combined with <code>Atomics</code> , they enable lock-free counters, mutexes, and condition variables.
Accept control	<code>nginx.suspendAcceptance()</code> / <code>nginx.suspendAllWorkers()</code> — pause connection acceptance during a mutation window, then resume.
Resource limits	<code>nginx.workerMemoryLimit</code> and <code>nginx.workerRequestTimeout</code> cap heap usage and CPU time per request; JS exceptions are caught; the worker survives.

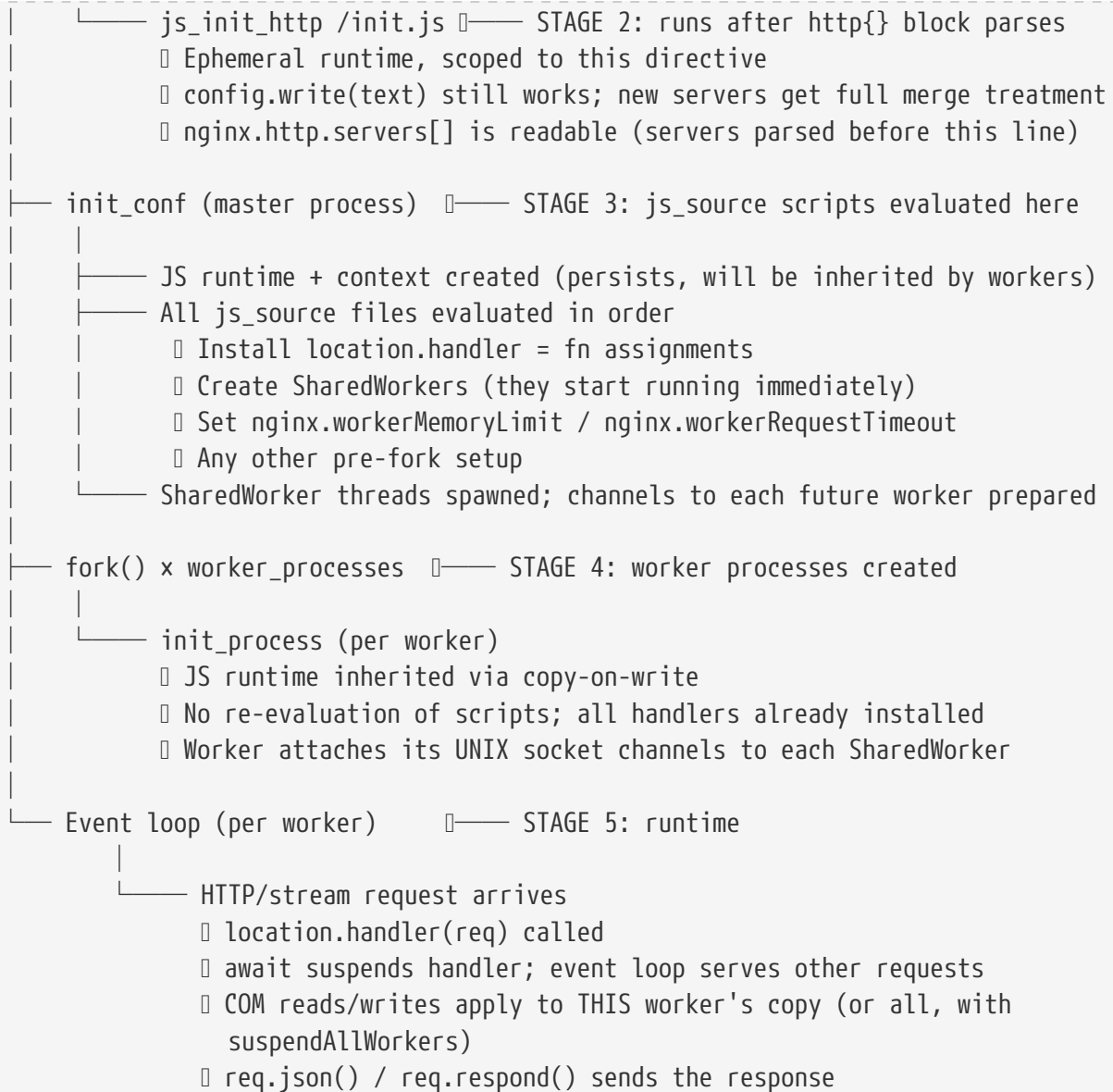
2. Architecture

2.1. Lifecycle Overview

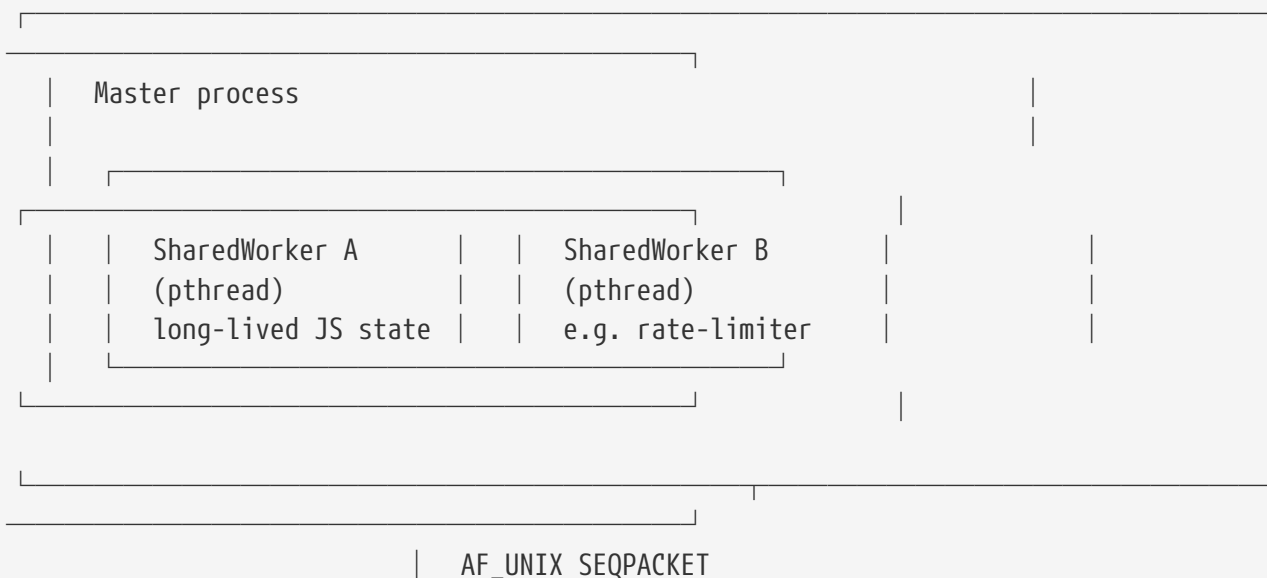
```

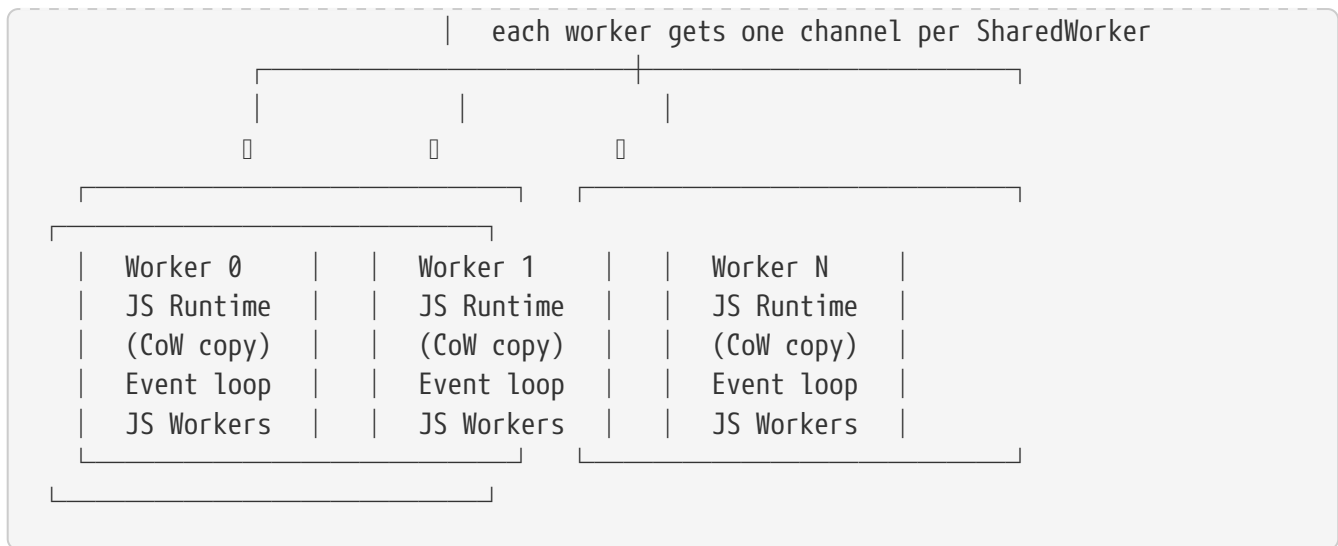
nginx start
|
├── Config file parsing begins (ngx_conf_parse)
|   |
|   ├── js_preprocess /gen.js ─── STAGE 1: runs immediately, mid-parse
|   |   ├── Ephemeral JS runtime (freed after script returns)
|   |   ├── config.write(text) injects text back into the parser
|   |   └── nginx.http, nginx.stream, etc. do NOT exist yet
|   |
|   └── ... rest of nginx.conf parsed (servers, locations, upstreams) ...

```



2.2. Process and Thread Topology





Key properties of this topology:

- Each nginx **worker process** has its own independent JS runtime. QuickJS is not thread-safe; no runtime is ever shared between processes.
- Workers inherit the master's runtime via copy-on-write after `fork()`. Scripts do not need to be re-evaluated; all handler functions, SharedWorker references, and global variables are ready immediately.
- **SharedWorkers** run in the master process as pthreads, outside any worker. Each worker communicates with a SharedWorker over a dedicated UNIX socket channel. This is the only channel through which workers can share mutable JS state.
- **JS Workers** (lowercase) are pthreads spawned **inside** a worker process. They exist to offload work from the event loop and are private to their nginx worker.

2.3. COM Ownership and Mutation Scope

All COM mutations go through one of three scopes:

Scope	Mechanism	Visible to
Per-request snapshot	<code>r.location.setWriteMode("local")</code> — writes a per-request copy of the location config	This request only; reversed when the request pool is freed
This worker only	Direct COM write (default)	All subsequent requests on this worker process
All workers atomically	<code>await nginx.suspendAllWorkers()</code> → mutate → <code>await nginx.resumeAllWorkers()</code>	All workers, once they resume

3. Quick Start

This section goes from "hello world" to "live upstream rebalancing" in four steps, each building on the previous one.

3.1. Hello World Handler

```
# nginx.conf
js_source /etc/nginx/js/init.js;

events { }

http {
    server {
        listen 80;
        location /hello { }    # placeholder – handler installed from JS
    }
}
```

```
// /etc/nginx/js/init.js
// This file is evaluated once in the master process after config parsing.
// All location.handler assignments made here are inherited by every worker.

const loc = nginx.http.servers[0].locations.find(l => l.path === '/hello');

loc.handler = (req) => {
    // req is a NginxRequest object. text() is a convenience shorthand
    // for respond(200, {'Content-Type': 'text/plain'}, body).
    req.text('Hello from JS DOM!\n');
};
```

3.2. Reading Request Data

```
// Read everything you might want from an incoming request.

loc.handler = async (req) => {
    // Basic identity
    const method = req.method;           // "GET", "POST", ...
    const uri     = req.uri;              // "/hello"
    const args    = req.args;             // raw query string: "a=1&b=2"
    const params  = req.queryParams;      // {a: "1", b: "2"}
    const scheme  = req.scheme;           // "http" or "https"
    const host    = req.host;             // "example.com"

    // Network info
    const client  = req.remoteAddr;       // "203.0.113.42"
    const port    = req.remotePort;       // 54321
    const server  = req.serverAddr;       // "192.0.2.1"

    // Headers (read-only object; keys are lowercased)
    const ua      = req.headers['user-agent'];
    const ct      = req.contentType;     // shorthand for content-type header
```

```
// Cookies: {name: "value", ...}
const session = req.cookies['session_id'];

// nginx variables – readable and writable via req.variables
const xff      = req.variables['http_x_forwarded_for'];
req.variables['my_custom_var'] = 'set-from-js';

// Request body (requires await – nginx reads it asynchronously)
const body     = await req.readBody(); // returns string

req.json({
  method, uri, client, ua, session,
  bodyLength: body.length
});
};
```

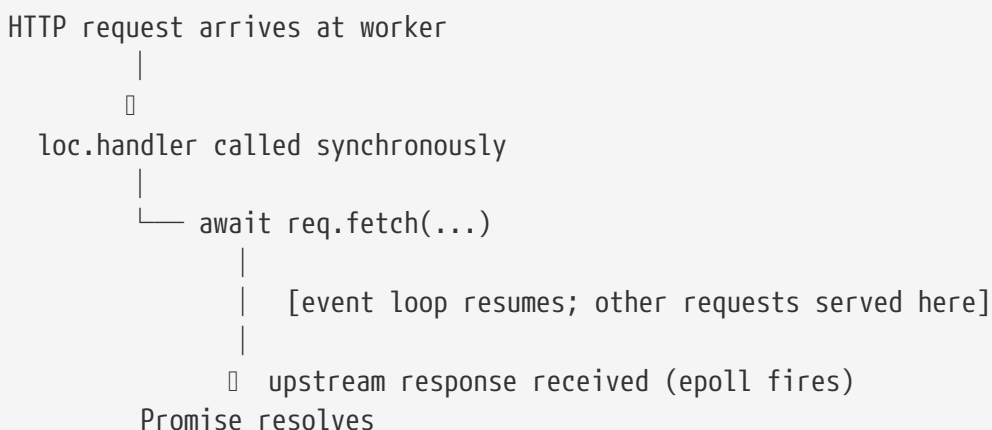
3.3. Async Fetch and Transform

```
// Non-blocking upstream call. While this handler awaits, the nginx event
// loop freely serves other requests on this worker.

loc.handler = async (req) => {
  // req.fetch() performs an HTTP subrequest through nginx's upstream
  // machinery. It returns a Promise that resolves with a Response object.
  const res = await req.fetch('http://api.internal/v1/prices');
  const data = await res.json();

  // Apply transformation
  const adjusted = data.items.map(item => ({
    ...item,
    price: (item.price * 1.10).toFixed(2) // add 10% margin
  }));

  req.json({ items: adjusted });
};
```



```

    |
    □
  res.json() parsed
    |
    □
  req.json(result) → HTTP response sent

```

3.4. Live Upstream Rebalancing

```

// An admin endpoint that atomically updates upstream peer weights across
// all workers – no nginx restart, no dropped connections.

const adminLoc = nginx.http.servers
  .find(s => s.names.includes('admin.internal'))
  .locations.find(l => l.path === '/upstream/rebalance');

adminLoc.handler = async (req) => {
  const body    = await req.readBody();
  const weights = JSON.parse(body);    // { "192.0.2.10:8080": 5, "192.0.2.11:8080":
10 }

  // Step 1: pause all workers so the change is seen atomically.
  await nginx.suspendAllWorkers();

  // Step 2: apply weights.
  const peers = nginx.http.upstreams
    .find(u => u.name === 'backend').peers;

  for (const peer of peers) {
    if (weights[peer.addr] !== undefined) {
      peer.weight = weights[peer.addr];
    }
  }

  // Step 3: resume – all workers pick up the new weights immediately.
  await nginx.resumeAllWorkers();

  req.json({ applied: true, peers: peers.map(p => ({addr: p.addr, weight:
p.weight})) });
};

```

4. Request Handlers

4.1. Installing a Handler

A location handler is a JavaScript function assigned to `location.handler` during `js_source` evaluation

(init_conf time):

```
nginx.http.servers[0].locations
  .find(l => l.path === '/api')
  .handler = async (req) => { ... };
```

The function receives one argument — a **NginxRequest** object — and may be synchronous or **async**. A location that has no JS handler falls through to nginx's normal content processing (proxy, static files, etc.).

4.2. NginxRequest Reference

4.2.1. Identity and Network

Property	Type	Description
method	string	HTTP method: "GET", "POST", "DELETE", ...
uri	string	Request URI without query string
args	string	Raw query string ("a=1&b=foo")
queryParams	object	Query string parsed to key → value pairs
scheme	string	"http" or "https"
host	string	Value of the Host header
remoteAddr	string	Client IP address
remotePort	number	Client TCP port
serverAddr	string	Server-side IP the connection arrived on
serverPort	number	Server-side port
httpVersion	string	"1.0", "1.1", or "2.0"
keepalive	boolean	Whether the connection is keep-alive
isInternal	boolean	true for subrequests
startTime	number	Request start time (milliseconds, Unix epoch)
requestLength	number	Total bytes received (headers + body)
connection	number	nginx connection serial number

4.2.2. Headers

Property / Method	Type	Description
headers	object	Request headers; keys are lowercased
contentType	string	Shorthand for headers['content-type']
contentLength	number	Parsed Content-Length value

Property / Method	Type	Description
<code>getHeader(name)</code>	string	Get a response header already set
<code>setHeader(name, v)</code>	—	Set a response header
<code>removeHeader(name)</code>	—	Remove a response header

4.2.3. Body

Property / Method	Type	Description
<code>body</code>	string null	Request body if already buffered; <code>null</code> otherwise
<code>readBody()</code>	Promise<string>	Read the full request body (async)

4.2.4. Cookies and Variables

Property / Method	Type	Description
<code>cookies</code>	object	Cookie jar: <code>{name: "value", ...}</code>
<code>variables</code>	object	Exotic r/w proxy over nginx variables
<code>getVar(name)</code>	string	Read one nginx variable
<code>setVar(name, val)</code>	—	Set one nginx variable

4.2.5. Location and Server Context

Property	Type	Description
<code>location</code>	NginxLocation	The matched location's COM object (mutable within this request)
<code>upstream</code>	object null	Upstream peer info if request was proxied

4.2.6. Response

Method	Description
<code>respond(status, headers, body)</code>	Low-level response; sends status + headers + body string in one call
<code>json(data)</code>	Send JSON: status 200, <code>Content-Type: application/json</code>
<code>text(body)</code>	Send plain text: status 200, <code>Content-Type: text/plain</code>
<code>html(body)</code>	Send HTML: status 200, <code>Content-Type: text/html</code>
<code>redirect(url)</code>	Send HTTP 302 redirect
<code>sendfile(path)</code>	Serve a file from the filesystem using <code>sendfile(2)</code>
<code>writeHead(status, headers)</code>	Start a streaming response (headers only)
<code>write(chunk)</code>	Append a chunk to a streaming response

Method	Description
<code>finish()</code>	End a streaming response
<code>statusCode</code> (r/w)	Get or set the response status code

4.2.7. Async Utilities

Method	Description
<code>sleep(ms)</code>	Pause the handler for <code>ms</code> milliseconds (non-blocking)
<code>fetch(url [, opts])</code>	Make a non-blocking upstream HTTP request; returns <code>Promise<Response></code>
<code>subrequest(uri)</code>	Issue an nginx internal subrequest; returns <code>Promise<Response></code>

4.2.8. Logging

```
req.log(nginx.INFO, 'Processing request from ' + req.remoteAddr);
// Log levels: nginx.DEBUG, nginx.INFO, nginx.NOTICE, nginx.WARN,
//             nginx.ERR, nginx.CRIT, nginx.ALERT, nginx.EMERG
```

4.3. Async Handler Lifecycle

```
nginx worker event loop
├── accept() → new connection
├── HTTP request parsed
├── location matched → location.handler(req) called
│   ├── [sync path] req.json(data)
│   │   └── response buffered → written to socket → done
│   └── [async path] await something
│       ├── Handler suspended. Worker event loop continues:
│       │   ├── accepts new connections
│       │   ├── runs other handlers
│       │   └── processes timers and upstream responses
│       └── event fires (timer / upstream / Worker message)
│           └── Promise resolves; handler resumes from the await point
│               └── req.json(data) → done
└── (next event)
```



Multiple async handlers can be suspended concurrently on the same worker. Each nginx worker runs a single-threaded event loop; async handlers cooperate by yielding at every **await** point, allowing other requests to be accepted and served in between. Use Workers or SharedWorkers when you need true CPU parallelism or need to share mutable state across workers.

4.4. Streaming Responses

```
loc.handler = async (req) => {
  // Send status + headers, then stream the body in chunks.
  req.writeHead(200, {'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache'});

  for (let i = 0; i < 5; i++) {
    req.write(`data: event ${i}\n\n`);
    await req.sleep(200);    // 200 ms between events, non-blocking
  }

  req.finish();
};
```

4.5. Subrequest Fan-Out

```
// Fire three upstreams in parallel and merge results.
loc.handler = async (req) => {
  const [a, b, c] = await Promise.all([
    req.subrequest('/internal/prices'),
    req.subrequest('/internal/stock'),
    req.subrequest('/internal/ratings'),
  ]);

  req.json({
    prices: await a.json(),
    stock:  await b.json(),
    ratings: await c.json(),
  });
};
```

4.6. Header and Body Filters

Filters are JavaScript functions that run in the nginx filter chain. They are installed on a location object:

```
// Header filter – modify response headers before they leave nginx
loc.headerFilter = (req, headers) => {
```

```

headers['X-Powered-By'] = 'NGINX JS DOM';
delete headers['Server'];
return headers;
};

// Body filter – transform response body chunks
loc.bodyFilter = (req, chunk, isLast) => {
  // Replace "foo" with "bar" in every body chunk
  return chunk.replace(/foo/g, 'bar');
};

```

Filters run in priority order (lower number = earlier in chain, default priority 50).

5. Configuration Object Model (COM)

5.1. Overview

The COM is a live JavaScript object tree rooted at the global `nginx` object. Properties read from the tree return current nginx struct values; writes (on mutable properties) take immediate effect.

```

nginx
├── version          – string  "1.27.x"
├── cpuCount         – number  (number of CPU cores)
├── workerMemoryLimit – number  (bytes; 0 = unlimited)  ← r/w
├── workerRequestTimeout – number (ms; 0 = unlimited)  ← r/w
├── log(level, msg)   – write to nginx error log
├── broadcast(msg)    – send msg to all workers' nginx.onbroadcast handler
├── suspendAcceptance() – per-worker accept pause (sync)
├── resumeAcceptance() – per-worker accept resume (sync)
├── suspendAllWorkers() – cross-worker quiesce (async, returns Promise)
├── resumeAllWorkers() – cross-worker resume (async, returns Promise)
├── cycle
│   ├── connections    – number (active connections)
│   ├── freeConnections – number
│   ├── openFilesN     – number
│   └── sockets[]      – listener socket objects
├── events
│   ├── workerConnections – number ← r/w
│   ├── multiAccept       – boolean ← r/w
│   └── acceptMutex       – boolean ← r/w
├── http
│   └── servers[]         – array of NginxServer (see below)

```

		upstreams[]	- array of NginxUpstream
		variables{}	- nginx HTTP variables
		serverNamesHashMaxSize	← r/w
		serverNamesHashBucketSize	← r/w
		variablesHashMaxSize	← r/w
		variablesHashBucketSize	← r/w
		addServer(name)	- create a new virtual server at runtime
		removeServer(name)	- remove a virtual server at runtime
		rebuildVhostDispatch()	- recompile the server_name dispatch hash
		match(req, conditions)	- evaluate an 'if' condition against a request
		sockets[]	- listening socket objects
		stream	
		servers[]	- array of stream NginxServer
		upstreams[]	- array of stream NginxUpstream

5.2. NginxServer

Each element of `nginx.http.servers[]` represents a `server {}` block.

```
const srv = nginx.http.servers[0];

// Identification
srv.name      // primary server_name
srv.names     // array of all server_names
srv.listen    // array of listen address strings

// Child collections
srv.locations // array of NginxLocation
srv.upstreams // upstreams referenced from this server (if any)

// SSL (if ssl module is compiled in)
srv.ssl.cert  // path to certificate ← r/w
srv.ssl.key   // path to private key ← r/w
srv.ssl.protocols // "TLSv1.2 TLSv1.3" ← r/w
srv.ssl.ciphers // cipher list ← r/w
srv.ssl.clientCert // path to CA cert for client auth ← r/w

// Dynamic location management
const loc = srv.addLocation('/new-path'); // returns NginxLocation
const loc = srv.addLocation('/api', { template: '/existing-path' }); // copy settings
                                                                    // from existing
loc

// Stream handler (stream {} servers only)
srv.handler = async (session) => { ... };
```

5.3. NginxLocation

```
const loc = nginx.http.servers[0].locations.find(l => l.path === '/api');

// Identification
loc.path          // "/api"
loc.exact         // true for "= /api" locations
loc.regex         // true for "~ /pattern" locations

// Handler
loc.handler = async (req) => { ... };

// Common directives (r/w)
loc.root          // document root path
loc.alias         // alias path
loc.index         // index file list
loc.tryFiles      // try_files argument list
loc.sendfile      // boolean
loc.gzip          // boolean
loc.gzipMinLength // number

// Proxy
loc.proxy.pass     // upstream URL ← r/w
loc.proxy.setHeader // {name: value} pairs ← r/w
loc.proxy.readTimeout // ms ← r/w

// Rate limiting, access, auth, headers – all module configs accessible
// (see §14 Reference for full list)

// Per-request write scoping
loc.setWriteMode("global") // write to global config (all future requests)
loc.setWriteMode("local")  // write only to a per-request snapshot
loc.setWriteMode("both")   // write to both (default)
loc.setReadMode("global")  // reads come from global config
loc.setReadMode("local")   // reads come from the snapshot (default)
```

5.4. NginxUpstream and Peers

```
const up = nginx.http.upstreams.find(u => u.name === 'backend');

up.name          // "backend"
up.peers         // array of NginxRRPeer

const p = up.peers[0];
p.addr          // "192.0.2.10:8080"
p.weight        // number ← r/w (takes a write lock internally)
p.maxFails      // number ← r/w
p.failTimeout   // ms ← r/w
```

```

p.down          // boolean ← r/w (marks peer as permanently down)
p.backup        // boolean (read-only)
p.conns         // number (active connections, live stat)
p.fails         // number (fail counter, live stat)

```

5.5. Iterating the Full Config Tree

```

// Dump the entire server/location tree to the error log.
for (const srv of nginx.http.servers) {
  nginx.log(nginx.INFO, `Server: ${srv.names.join(', ')}\`);
  for (const loc of srv.locations) {
    nginx.log(nginx.INFO, ` ${loc.path} handler=${loc.handler ? 'JS' :
'nginx'}\`);
  }
}

// Find all locations that use a specific upstream
const byUpstream = nginx.http.servers
  .flatMap(s => s.locations)
  .filter(l => l.proxy?.pass?.includes('backend'));

```

6. Dynamic Reconfiguration

6.1. Scope 1 — This Worker Only

The simplest mutations happen directly in a handler with no coordination. The change takes effect immediately for all subsequent requests on this worker. Other workers retain their own CoW copy unchanged.

```

// Use-case: A/B flag toggle via admin endpoint, applied to this worker.
// Fine for experiments where eventual consistency across workers is acceptable.

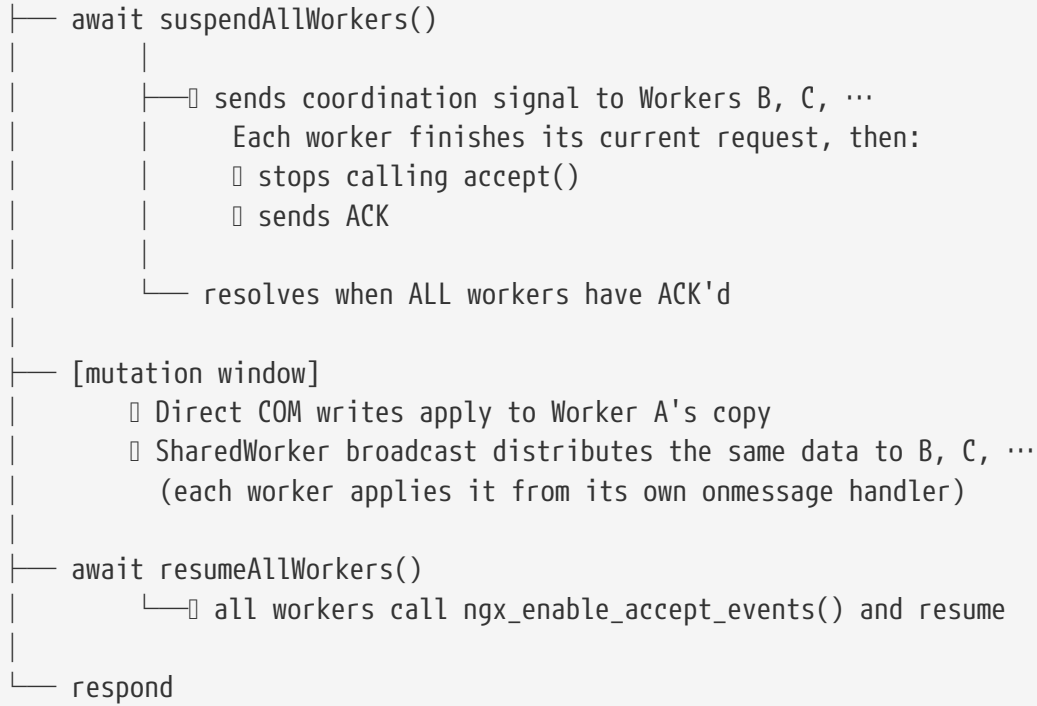
nginx.http.servers[0].locations
  .find(l => l.path === '/admin/toggle-gzip')
  .handler = (req) => {
    // Toggle gzip for the /api location on THIS worker only
    const apiLoc = nginx.http.servers[0].locations
      .find(l => l.path === '/api');
    apiLoc.gzip = !apiLoc.gzip;
    req.text(`gzip is now ${apiLoc.gzip}\`);
  };

```

6.2. Scope 2 — All Workers Atomically

For changes that must be consistent across every worker simultaneously, use `suspendAllWorkers` / `resumeAllWorkers`.

Worker A (handler)



```
adminLoc.handler = async (req) => {
  const newPeers = JSON.parse(await req.readBody());

  await nginx.suspendAllWorkers();

  // Apply to this worker
  const peers = nginx.http.upstreams[0].peers;
  newPeers.forEach((p, i) => {
    if (peers[i]) peers[i].weight = p.weight;
  });

  await nginx.resumeAllWorkers();

  req.json({ ok: true });
};
```

6.3. Scope 3 — Add / Remove Servers and Locations

`addServer`, `removeServer`, `addLocation` enable structural mutations — adding or removing routing entries at runtime.

```
// Tenant onboarding: create a new vhost live, no restart.

nginx.http.servers
  .find(s => s.names.includes('admin.internal'))
  .locations.find(l => l.path === '/tenant/create')
  .handler = async (req) => {
    const body = JSON.parse(await req.readBody());
    const domain = body.domain;

    await nginx.suspendAllWorkers();

    // Create the new server
    const srv = nginx.http.addServer(domain);
    srv.listen.push('80');

    // Add a root location with a JS handler
    const loc = srv.addLocation('/');
    loc.handler = (r) => {
      r.text(`Welcome to ${domain}!\n`);
    };

    // Rebuild the server_name dispatch hash so the new Host: domain routes
    // correctly to our new server.
    nginx.http.rebuildVhostDispatch();

    await nginx.resumeAllWorkers();

    req.json({ created: domain });
  };
};
```

```
// Remove a tenant server (e.g. account deleted).
nginx.http.removeServer('ex-tenant.example.com');
nginx.http.rebuildVhostDispatch();
```



`rebuildVhostDispatch()` must be called after any change to server names. Without it the Host-based routing hash is stale and requests will be routed to the wrong server.

6.4. Scope 4 — Per-Request Config Snapshot

The `location.setWriteMode("local")` API creates a copy of the location's config struct scoped to the current request. The global struct is untouched; other concurrent requests see the original values.

```
loc.handler = async (req) => {
  // Write only to a private snapshot for this request.
  req.location.setWriteMode('local');
```

```
// This change is visible only inside this handler invocation.
req.location.root = '/var/www/' + req.getVar('user_dir');
req.location.gzip = false;

// ... process request ...
// When the request pool is freed, the snapshot is discarded automatically.
};
```

6.5. Hot SSL Certificate Swap

```
// Admin handler: reload the TLS certificate from disk, zero downtime.
// New connections after this call use the new certificate;
// in-flight TLS sessions are unaffected.

adminLoc.handler = async (req) => {
  const body = JSON.parse(await req.readBody());
  const srv = nginx.http.servers.find(s => s.names.includes(body.domain));
  if (!srv) { req.respond(404, {}, 'unknown domain'); return; }

  await nginx.suspendAllWorkers();
  srv.ssl.cert = body.certPath;
  srv.ssl.key = body.keyPath;
  await nginx.resumeAllWorkers();

  req.json({ reloaded: body.domain });
};
```

7. Pre-Fork Initialization

7.1. `js_preprocess` — Config Injection During Parse

`js_preprocess` evaluates a JS script **during** `ngx_conf_parse`, at the exact point where the directive appears in the config file. The script receives a global `config` object with a single method:

`config.write(text)` — feeds `text` back into the nginx config parser exactly as if it had appeared inline at this position.

This happens **before** `init_conf`, so `nginx.http`, `nginx.stream`, and the full COM are not available. The `std` and `os` QuickJS modules are available for file and OS access.

```
nginx.conf parse reaches js_preprocess /gen.js
|
├─ New JS runtime created (isolated, short-lived)
|
└─ gen.js evaluated
```

- | □ Can read files: std.loadFile('/etc/conf.json')
- | □ Can read env: std.getenv('APP_ENV')
- | □ Can run commands: os.exec([...])
- | □ Can call config.write(text)
- |
- |— Runtime freed
- |
- |— Injected text re-enters the parser as if it were inline
 - full http{}/server{}/location{} blocks allowed
 - directives go through normal merge/optimize_servers treatment

7.1.1. Example: Server Blocks from a JSON File

```
// /etc/nginx/js/gen.js
// Generates one server {} block per tenant from a JSON config file.
// Runs once at startup (and again on `nginx -s reload`).

import * as std from 'std';

// Read tenant definitions from a JSON file.
const raw    = std.loadFile('/etc/nginx/tenants.json');
const tenants = JSON.parse(raw);

let out = '';
for (const t of tenants) {
  out += `
server {
  listen 80;
  server_name ${t.domain};
  root /var/www/${t.id};
  location / { }
}
`;
}

config.write(out);
```

```
# nginx.conf
js_preprocess /etc/nginx/js/gen.js;  # ← generates server{} blocks

events { }
http {
  # servers from gen.js appear here as if written manually
}
```

7.1.2. Example: Conditional SSL Config

```
// Enable SSL only if the certificate file actually exists.
import * as os from 'os';

const certPath = '/etc/ssl/certs/site.pem';
const keyPath = '/etc/ssl/private/site.key';

// os.stat() returns an object on success, error code on failure.
const [stat, err] = os.stat(certPath);
if (!err) {
    config.write(`
ssl_certificate      ${certPath};
ssl_certificate_key  ${keyPath};
listen 443 ssl;
`);
}
```

7.1.3. Example: Fetching Config from a Network Service

`js_preprocess` runs synchronously inside the C parse loop. There is no event loop at this stage, so the standard `req.fetch()` API is unavailable. The `std.urlGet()` function ([QuickJS std library](#)) works around this by spawning `curl` as a child process and blocking until the response arrives.

`curl` must be installed on the system.

```
// /etc/nginx/js/fetch-config.js
// Download upstream configuration from a config service at startup.
// Uses std.urlGet() which shells out to curl – blocking and synchronous.

import * as std from 'std';

const res = std.urlGet('http://config-service.internal/v1/nginx-upstreams',
    { full: true }); // { status, headers, body }

if (res.status !== 200) {
    // Fail loudly – a bad config is worse than a failed start.
    throw new Error(`Config service returned ${res.status}`);
}

const cfg = JSON.parse(res.body);

// Generate upstream{} blocks
let out = '';
for (const up of cfg.upstreams) {
    out += `upstream ${up.name} {\n`;
    for (const peer of up.peers) {
        out += `    server ${peer.addr} weight=${peer.weight};\n`;
    }
}
```

```
    out += `}\n`;
}
config.write(out);
```



`std.urlGet` always spawns a `curl` subprocess. For reliability in production, prefer writing config data to a file during a pre-start hook and reading it with `std.loadFile()`, keeping `js_preprocess` independent of network availability at start time. The network fetch pattern is useful for short-lived tooling or environments where the config service is guaranteed to be reachable.

7.1.4. Multiple `js_preprocess` Directives

Multiple `js_preprocess` directives are allowed; each runs in its own isolated runtime in declaration order:

```
js_preprocess /etc/nginx/js/gen-servers.js;    # generates server{} blocks
js_preprocess /etc/nginx/js/gen-upstreams.js;  # generates upstream{} blocks
js_preprocess /etc/nginx/js/gen-maps.js;       # generates map{} blocks

events { }
http { ... }
```

7.2. `js_init_http` — Post-Parse Server Injection

`js_init_http` fires after the entire `http {}` block has been parsed but before directive merging takes place. It provides both `config.write()` and a read-only view of `nginx.http.servers[]` — the servers parsed before the `js_init_http` directive.

```
nginx.conf
├── http {
│   ├── server { server_name primary.example.com; ... }
│   ├── server { server_name secondary.example.com; ... }
│   └── js_init_http /etc/nginx/js/post-parse.js;
│       ├── nginx.http.servers[0] and [1] are readable
│       ├── config.write(text) injects new servers after these two
│       └── injected servers go through normal merge/init_locations treatment
```

```
// /etc/nginx/js/post-parse.js
// Add a management server on port 8099 only if no server is already
// listening on that port.
```

```
import * as std from 'std';

// Check whether any existing server listens on 8099
const already = nginx.http.servers.some(s =>
  s.listen.some(addr => addr.includes(':8099') || addr === '8099')
);

if (!already) {
  config.write(`
server {
  listen 8099;
  server_name management.internal;

  location /health { }
  location /metrics { }
}
`);
}
```



Place `js_init_http` **after** the `server {}` blocks it needs to read. Servers parsed after the directive are not visible to the script.

8. Workers (JS Web Workers)

JS Workers are background pthreads spawned inside an nginx worker process. They implement the standard [Web Worker API](#) and are the correct tool when a request needs to perform CPU-intensive computation without blocking the event loop.

Each nginx worker process has its own set of Workers; they are not shared between nginx workers.

8.1. Basic Worker Pattern

```
// init.js (js_source)

// Spawn a Worker thread for CPU-intensive prime computation.
// Workers are created once in js_source and reused across all requests.
const computeWorker = new Worker('/etc/nginx/js/prime_worker.js');

nginx.http.servers[0].locations
  .find(l => l.path === '/primes')
  .handler = async (req) => {
    const n = parseInt(req.queryParams.n) || 100;

    // Send work to the Worker thread and await the result.
    // While awaiting, the event loop is free to serve other requests.
    const result = await new Promise(resolve => {
```

```

        computeWorker.onmessage = e => resolve(e.data);
        computeWorker.postMessage({ n });
    });

    req.json(result);
};

```

```

// /etc/nginx/js/prime_worker.js – runs in a separate pthread

function sieve(n) {
    const composite = new Uint8Array(n + 1);
    for (let i = 2; i * i <= n; i++) {
        if (!composite[i]) {
            for (let j = i * i; j <= n; j += i) composite[j] = 1;
        }
    }
    return Array.from({length: n - 1}, (_, i) => i + 2).filter(i => !composite[i]);
}

// Worker message loop
onmessage = (e) => {
    const primes = sieve(e.data.n);
    postMessage({ count: primes.length, largest: primes[primes.length - 1] });
};

```

```

HTTP request → nginx worker event loop
|
|— handler called; sends postMessage to Worker thread
|
|   □
|   Worker pthread
|   (independent JS runtime)
|   sieve(n) ← CPU-bound, runs without blocking event loop
|
|   □
|   postMessage(result)
|
|   □ onmessage event delivered to event loop
|
|— Promise resolves; req.json(result) → response sent

```

8.2. Transferring ArrayBuffer (Zero Copy)

Passing an **ArrayBuffer** by transfer moves ownership; the sender can no longer access it. Use this for large binary payloads:

```
// Compress a large buffer in a Worker thread, transfer result back.
const compressionWorker = new Worker('/etc/nginx/js/compress_worker.js');

loc.handler = async (req) => {
  const body = await req.readBody();
  const input = new TextEncoder().encode(body).buffer; // ArrayBuffer

  const compressed = await new Promise(resolve => {
    compressionWorker.onmessage = e => resolve(e.data.buf);
    // Transfer 'input' – zero copy, Worker takes ownership
    compressionWorker.postMessage({ buf: input }, [input]);
  });

  req.setHeader('Content-Encoding', 'deflate');
  req.respond(200, {}, compressed);
};
```

8.3. Worker **std** and **os** Modules

Workers have access to the QuickJS standard library:

```
// worker.js
import * as std from 'std';
import * as os from 'os';

onmessage = (e) => {
  // Blocking file read – safe in a Worker (doesn't block the event loop)
  const data = std.loadFile(e.data.path);
  postMessage({ content: data, length: data?.length ?? 0 });
};
```

See [QuickJS Standard Library](#) for the full **std/os** API reference.

9. SharedWorkers

9.1. Concept and Use Cases

A **SharedWorker** is a single long-lived pthread running in the nginx **master** process, connected to every nginx worker via dedicated UNIX socket channels. Unlike JS Workers (one per nginx worker), there is exactly one SharedWorker instance regardless of the **worker_processes** count.

This makes SharedWorkers the canonical location for:

- **Session stores** — a token → user mapping visible to every worker.
- **Distributed rate limiters** — accurate global request-per-second counting.

- **Feature flags** — a single flag store; updates propagate to all workers on the next message.
- **Distributed locks** — a central lock arbiter that any worker can acquire.
- **Config broadcast** — push a config change to all workers simultaneously (see [\[scope-2-all-workers-atomically\]](#)).

9.2. Port-Based Pattern (Explicit **onconnect**)

Use this pattern when a worker needs a dedicated two-way channel with the SharedWorker (i.e. replies must go back to the same worker that sent the request):

```
// /etc/nginx/js/session_sw.js – SharedWorker script

const store = new Map(); // token → {userId, expires}

// onconnect fires once per nginx worker when the channel is first used.
onconnect = (e) => {
  const port = e.ports[0];

  port.onmessage = (msg) => {
    const { cmd, token, data } = msg.data;

    if (cmd === 'get') {
      const entry = store.get(token) ?? null;
      // Reply only to the worker that asked
      port.postMessage({ token, entry });
    } else if (cmd === 'set') {
      store.set(token, { userId: data.userId,
                        expires: Date.now() + data.ttl });
      port.postMessage({ ok: true });
    } else if (cmd === 'del') {
      store.delete(token);
      port.postMessage({ ok: true });
    }
  };
};
```

```
// init.js (js_source)

const sessionSW = new SharedWorker('/etc/nginx/js/session_sw.js');

// Helper: promise-based call into the SharedWorker.
function swCall(cmd, token, data) {
  return new Promise(resolve => {
    sessionSW.port.onmessage = e => resolve(e.data);
    sessionSW.port.postMessage({ cmd, token, data });
  });
}
```

```

});
}

nginx.http.servers[0].locations
  .find(l => l.path === '/auth')
  .handler = async (req) => {
    const token = req.cookies['session'];
    if (!token) { req.respond(401, {}, 'no session'); return; }

    const { entry } = await swCall('get', token, null);
    if (!entry || entry.expires < Date.now()) {
      req.respond(401, {}, 'invalid or expired session');
      return;
    }

    req.setVar('authenticated_user', entry.userId);
    req.respond(200, {}, `Hello ${entry.userId}`);
  };

```

9.3. Broadcast Pattern (Global **onmessage** + **clients**)

Use this pattern when the SharedWorker needs to push the same message to **all** connected workers simultaneously:

```

// /etc/nginx/js/broadcast_sw.js

// No onconnect – global onmessage handles all workers via the `clients` array.
// `clients` is an array of all currently connected port objects.

onmessage = (msg) => {
  if (msg.data.cmd === 'broadcast') {
    // Forward the message to every connected worker
    clients.forEach(c => c.postMessage(msg.data.payload));
  }
};

```

```

// init.js: trigger a broadcast from any worker's request handler.
const broadcastSW = new SharedWorker('/etc/nginx/js/broadcast_sw.js');

// Each worker registers a listener on the SW port at init time.
broadcastSW.port.onmessage = (e) => {
  // This fires on every worker when a broadcast arrives.
  const { key, value } = e.data;
  // Apply the change to this worker's local copy of the config.
  applyFeatureFlag(key, value);
};

// Admin handler: update a feature flag on all workers at once.

```

```

nginx.http.servers[0].locations
  .find(l => l.path === '/admin/feature-flag')
  .handler = async (req) => {
    const { key, value } = JSON.parse(await req.readBody());

    // Tell the SharedWorker to broadcast to all workers.
    broadcastSW.port.postMessage({
      cmd: 'broadcast',
      payload: { key, value }
    });

    req.json({ ok: true });
  };

```

9.4. Dynamic SharedWorker Creation

SharedWorkers created in `js_source` are spawned before `fork()` and are available to all workers from the start. They can also be created dynamically from a request handler after `fork()`:

```

// Create a SharedWorker on demand from a handler.
// The SharedWorker is registered globally and subsequent handlers
// calling new SharedWorker('/js/dynamic.js') reuse the same thread.

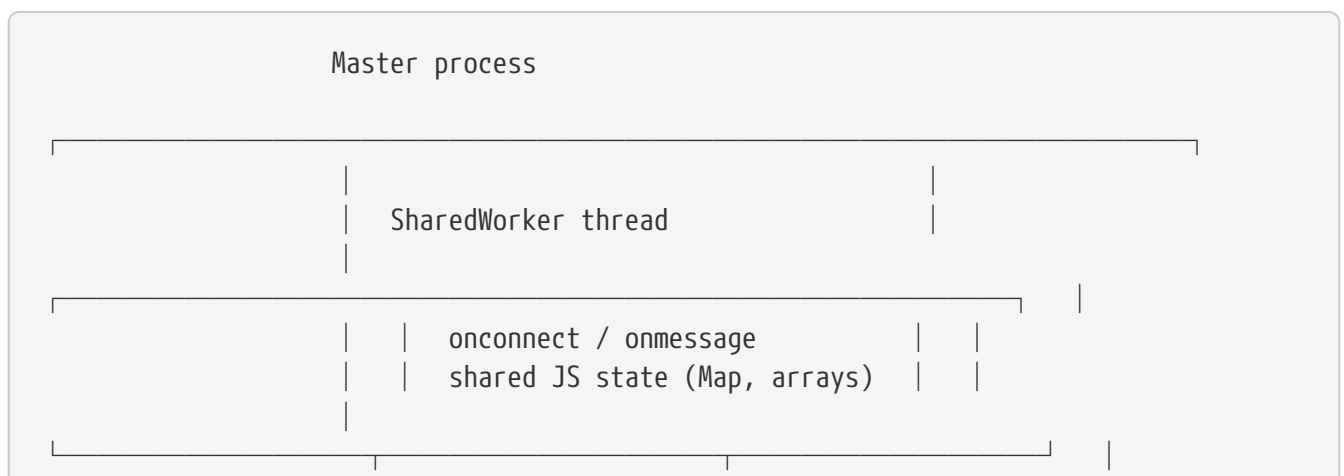
loc.handler = async (req) => {
  const sw = new SharedWorker('/etc/nginx/js/on_demand_sw.js');

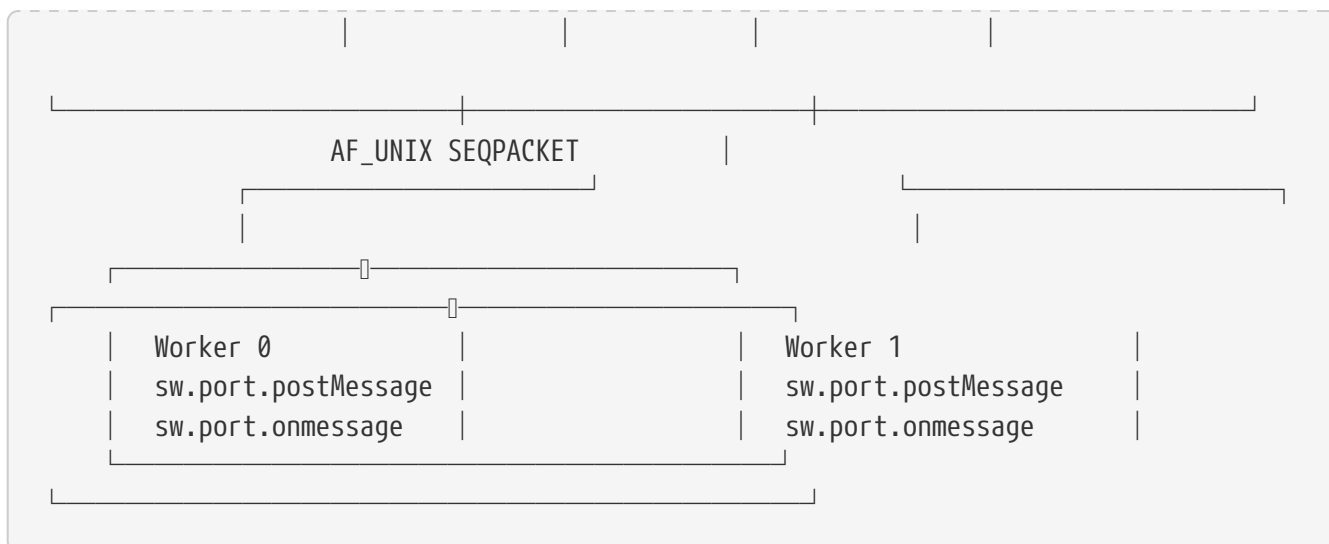
  const result = await new Promise(resolve => {
    sw.port.onmessage = e => resolve(e.data);
    sw.port.postMessage({ req: req.uri });
  });

  req.json(result);
};

```

9.5. SharedWorker Flow Chart





10. SharedArrayBuffers and Atomics

10.1. What They Are

A **SharedArrayBuffer** is a region of memory accessible — with zero copying — from any thread: nginx workers, JS Workers, and the SharedWorker thread.

Typed array views (**Int32Array**, **Float64Array**, **BigInt64Array**, etc.) over a SAB provide direct read/write access. **Atomics** operations (**add**, **compareExchange**, **wait**, **notify**) provide the synchronisation primitives needed for safe concurrent access.

10.2. Pre-Fork SABs (Visible to All Workers)

A SAB created in **js_source** is backed by the CoW page mapping inherited by every worker after **fork()**. All workers share the same physical pages.

```
// init.js (js_source)
// A global hit counter visible to all workers with zero copy.

const counterBuf = new SharedArrayBuffer(8);
const hits      = new BigInt64Array(counterBuf);
// hits[0] is shared by every worker process.

nginx.http.servers[0].locations
  .find(l => l.path === '/count')
  .handler = (req) => {
    // Atomic increment – safe across all nginx workers simultaneously.
    const n = Atomics.add(hits, 0, 1n);
    req.text(`Request #${n + 1n}\n`);
  };

nginx.http.servers[0].locations
  .find(l => l.path === '/stats')
```

```
.handler = (req) => {
  req.json({ total_hits: Number(Atomics.load(hits, 0)) });
};
```

10.3. Post-Fork SABs (Worker-Local, Passed via Channels)

A SAB created inside a request handler (post-fork) is backed by a Linux `memfd` and can be transferred to a JS Worker or SharedWorker via `postMessage`. The receiving thread `mmap`'s the same file descriptor, establishing a shared mapping without copying data.

```
// Worker thread computes a result into a SAB; main thread reads it.

const processorWorker = new Worker('/etc/nginx/js/processor.js');

loc.handler = async (req) => {
  // Allocate shared memory for the result
  const resultBuf = new SharedArrayBuffer(1024);
  const result    = new Uint8Array(resultBuf);
  const ready     = new SharedArrayBuffer(4);
  const flag      = new Int32Array(ready);

  // Pass both SABs to the Worker; it writes the result and sets flag.
  processorWorker.postMessage({
    input:    JSON.parse(await req.readBody()),
    resultBuf, // shared – Worker writes here
    ready     // shared – Worker sets flag[0]=1 when done
  });

  // Non-blocking poll using Atomics.wait in the Worker thread
  // Meanwhile the event loop remains free.
  await new Promise(resolve => {
    processorWorker.onmessage = () => resolve();
  });

  req.respond(200, {}, new TextDecoder().decode(result));
};
```

10.4. Atomics.wait / Atomics.notify

`Atomics.wait` is a **blocking** call — it must only be used in Worker or SharedWorker threads, never in the main nginx event loop.

```
// Mutex implementation using SAB + Atomics.
// Safe to use in Worker and SharedWorker threads.
```

```

const mutexBuf = new SharedArrayBuffer(4); // one Int32 = one mutex
const mutex    = new Int32Array(mutexBuf);

function lock() {
  // Spin until we CAS 0 → 1 (acquire)
  while (Atomics.compareExchange(mutex, 0, 0, 1) !== 0) {
    // Block this thread until mutex[0] changes – woken by unlock()
    Atomics.wait(mutex, 0, 1);
  }
}

function unlock() {
  Atomics.store(mutex, 0, 0);
  // Wake one waiting thread
  Atomics.notify(mutex, 0, 1);
}

```

```

// In the SharedWorker thread – use Atomics.wait/notify to signal workers.
// SharedWorker receives work, signals the waiting nginx worker via SAB.

onconnect = (e) => {
  const port = e.ports[0];

  port.onmessage = (msg) => {
    const { sab, workItem } = msg.data;
    const signal = new Int32Array(sab);

    // Do the work ...
    const result = processWork(workItem);

    // Write result into a different part of the SAB, then notify.
    // The nginx worker is blocked in Atomics.waitAsync on this index.
    Atomics.store(signal, 0, 1); // mark done
    Atomics.notify(signal, 0);   // wake the nginx worker's Worker thread
  };
};

```



Atomics.waitAsync (non-blocking, returns a Promise) is available in the nginx event loop context and is the correct way to wait for a SAB value to change from the main thread.

11. Accept Control and Cross-Worker Coordination

11.1. The Problem

COM mutations applied directly from a handler affect only the worker that executed the handler. Other workers have independent CoW copies and will not see the change. Furthermore, if a mutation is applied while another request on the same worker is halfway through using the old value, the result can be inconsistent.

Accept control solves both problems by temporarily pausing connection acceptance, creating a window during which no new requests can start and the worker is in a known-clean state.

11.2. Per-Worker: `suspendAcceptance` / `resumeAcceptance`

These are synchronous, instant, and affect only the calling worker.

```
loc.handler = (req) => {  
  // Stop accepting new connections on THIS worker.  
  // OS backlog holds new connections; other workers keep accepting normally.  
  nginx.suspendAcceptance();  
  
  // Apply a batch of mutations atomically from this worker's perspective.  
  const upstream = nginx.http.upstreams[0];  
  upstream.peers[0].weight = 10;  
  upstream.peers[1].weight = 5;  
  upstream.peers[2].down   = true;  
  
  // Resume accepting.  
  nginx.resumeAcceptance();  
  
  req.text('upstream updated\n');  
};
```

What happens while suspended:

```
OS kernel  
|  
└── TCP SYN backlog accumulates (up to net.core.somaxconn connections)  
    |  
    └── When resumeAcceptance() is called, epoll fires again and  
        the worker drains the backlog in FIFO order.
```

Other workers are unaffected and continue accepting normally.

11.3. Cross-Worker: `suspendAllWorkers` / `resumeAllWorkers`

For changes that must be atomic across all workers simultaneously:

Worker A (handler)

```
|
|— await nginx.suspendAllWorkers()
|   |
|   |— Workers B, C, ... receive coordination signal
|   |   Each finishes current request → stops accepting → ACKs
|   |— resolves when all have ACK'd
|
|— mutation window (all workers quiesced)
|
|— await nginx.resumeAllWorkers()
|   |— all workers re-add listening sockets to epoll
|
|— req.respond(...)
```

11.4. SharedWorker Broadcast + Accept Control Pattern

The most flexible pattern for cross-worker config updates:

```
// coord_sw.js – SharedWorker for cross-worker coordination

const ports = []; // one port per connected worker

onconnect = (e) => {
  const port = e.ports[0];
  ports.push(port);

  port.onmessage = (msg) => {
    if (msg.data.cmd === 'broadcast_apply') {
      // Push the same config update to every worker
      ports.forEach(p => p.postMessage({
        cmd: 'apply',
        config: msg.data.config
      }));
    }
  };
};
```

```
// init.js
const coordSW = new SharedWorker('/etc/nginx/js/coord_sw.js');

// Each worker registers a listener to apply inbound config changes.
coordSW.port.onmessage = (e) => {
  if (e.data.cmd === 'apply') {
    applyConfig(e.data.config);
  }
};
```

```

        nginx.resumeAcceptance();
    }
};

function applyConfig(cfg) {
    // Apply upstream changes to this worker's local copy.
    const peers = nginx.http.upstreams[0].peers;
    cfg.peers.forEach((p, i) => {
        if (peers[i]) {
            peers[i].weight = p.weight;
            peers[i].down    = p.down;
        }
    });
}

// Admin endpoint: one worker drives the update, broadcasts to all others.
nginx.http.servers[0].locations
    .find(l => l.path === '/admin/config-push')
    .handler = async (req) => {
        const cfg = JSON.parse(await req.readBody());

        // Quiesce all workers, apply locally, then broadcast.
        await nginx.suspendAllWorkers();

        applyConfig(cfg); // this worker applies locally

        // Tell SharedWorker to push to all workers; they will call
        // resumeAcceptance() from their onmessage handler.
        coordSW.port.postMessage({ cmd: 'broadcast_apply', config: cfg });

        // This worker also resumes acceptance.
        nginx.resumeAcceptance();

        req.json({ pushed: true });
    };
};

```

11.5. Backlog Behaviour

While any or all workers have acceptance suspended, incoming TCP connections queue in the OS SYN/accept backlog:

- Backlog size is controlled by `net.core.somaxconn` (typically 128–4096) and the `listen` directive's `backlog=N` parameter.
- If the backlog fills before `resumeAcceptance()` is called, the kernel drops new SYN packets. From the client's perspective this looks like the server is temporarily unavailable.
- For short mutation windows (sub-millisecond), the probability of backlog overflow is negligible even under high load.



Do not hold all workers suspended for long operations (database queries, external HTTP calls). Complete all blocking work before calling `suspendAllWorkers()`, then apply changes synchronously in the mutation window.

12. Snapshots

Snapshots capture a point-in-time copy of a COM sub-tree. They are useful for rollback, for auditing config state, and for read-heavy operations that must be isolated from concurrent writes.

12.1. Basic Snapshot and Rollback

```
loc.handler = async (req) => {
  // Capture the current state of a location before mutating it.
  const snap = req.location.snapshot();

  try {
    // Attempt to apply new configuration.
    const cfg = JSON.parse(await req.readBody());
    req.location.gzip      = cfg.gzip;
    req.location.root      = cfg.root;
    req.location.proxy.pass = cfg.proxyPass;

    req.json({ applied: true });
  } catch (e) {
    // Something went wrong – restore the original state.
    snap.restore();
    req.respond(500, {}, `Config apply failed: ${e.message}`);
  }
};
```

12.2. Sub-Object Snapshots

You can snapshot a specific COM sub-tree rather than the entire config:

```
// Snapshot just the upstream peer list.
const snap = nginx.http.upstreams[0].snapshot();

// Apply potentially risky changes.
nginx.http.upstreams[0].peers.forEach(p => p.weight = 1);

// If health-check fails, restore peer weights.
const healthy = await runHealthCheck();
if (!healthy) {
  snap.restore();
}
```

12.3. Read / Write Mode Interaction with Snapshots

When `setWriteMode("local")` is active, COM writes go to a per-request snapshot automatically, leaving the global config untouched:

```
loc.handler = async (req) => {
  // Isolate all writes to this request.
  req.location.setWriteMode('local');
  req.location.setReadMode('local');

  // Mutate the per-request copy freely.
  req.location.root = '/var/www/' + req.getVar('tenant');

  // Other concurrent requests still see the original global root.
  req.sendfile('/index.html');
};
```

13. Stream (TCP/UDP) Support

The JS DOM API extends to nginx's `stream {}` context. Stream handlers receive a `NginxStreamSession` object instead of `NginxRequest`.

```
js_source /etc/nginx/js/stream_init.js;

events { }
```

```
stream {
  server {
    listen 9000;
  }
}
```

```
// stream_init.js

nginx.stream.servers[0].handler = function(session) {
  // Session properties
  session.remoteAddress // client IP
  session.remotePort    // client port
  session.localAddress  // server-side IP
  session.localPort     // server-side port (9000)
  session.ssl           // boolean
  session.received      // bytes received so far

  // Variable access (stream variables)
  const proto = session.variable('protocol'); // "TCP" or "UDP"
```

```
// Close the session with a status code
session.finalize();           // NGX_STREAM_OK
session.finalize(500);        // error
};
```

Stream COM mirrors the HTTP COM:

```
// Read stream upstream peers
const peers = nginx.stream.upstreams[0].peers;
peers.forEach(p => nginx.log(nginx.INFO, `${p.addr} w=${p.weight}`));

// Mutate stream upstream peers (same API as HTTP)
peers[0].weight = 20;
```

14. External Config Sources and Full Reload

14.1. Pattern A — File at Startup (**js_preprocess**)

Config data is written to a file by an external process (deploy script, Kubernetes operator, config management tool) and read synchronously during nginx startup:

```
// /etc/nginx/js/from-file.js
import * as std from 'std';

const cfg = JSON.parse(std.loadFile('/etc/nginx/dynamic-config.json') || '{}');

for (const vhost of (cfg.vhosts || [])) {
  config.write(`
server {
  listen 80;
  server_name ${vhost.domain};
  root /var/www/${vhost.docroot};
  location / { }
}
`);
}
```

This runs on every **nginx -s reload**, so the file is the single source of truth.

14.2. Pattern B — Network at Startup (**js_preprocess** + **std.urlGet**)

See the example in [\[example-fetching-config-from-a-network-service\]](#). Use this pattern for tooling environments where network availability is guaranteed. In production, pre-download to a file during a health-check or init-container step to avoid coupling nginx startup to network availability.

14.3. Pattern C — Live Reload via Handler (Zero Downtime)

For changes that do not require a full restart — upstream weights, SSL certificates, location handlers — apply mutations via the COM from a request handler:

```
// init.js

function applyConfig(cfg) {
  // Update upstream peer weights
  if (cfg.upstreams) {
    for (const [name, peerCfg] of Object.entries(cfg.upstreams)) {
      const up = nginx.http.upstreams.find(u => u.name === name);
      if (!up) continue;
      for (const [addr, w] of Object.entries(peerCfg)) {
        const peer = up.peers.find(p => p.addr === addr);
        if (peer) peer.weight = w;
      }
    }
  }

  // Hot-swap SSL certificates
  if (cfg.ssl) {
    for (const [domain, paths] of Object.entries(cfg.ssl)) {
      const srv = nginx.http.servers.find(s => s.names.includes(domain));
      if (!srv) continue;
      srv.ssl.cert = paths.cert;
      srv.ssl.key = paths.key;
    }
  }
}

// Admin endpoint: POST the new config JSON.
nginx.http.servers
  .find(s => s.names.includes('admin.internal'))
  .locations.find(l => l.path === '/config/apply')
  .handler = async (req) => {
    const cfg = JSON.parse(await req.readBody());

    await nginx.suspendAllWorkers();
    applyConfig(cfg);
    await nginx.resumeAllWorkers();

    req.json({ applied: true, ts: Date.now() });
  };

// Pull-based reload: a background Worker polls the config service
// periodically and applies changes.
const pollerWorker = new Worker('/etc/nginx/js/config_poller.js');
```

```
pollerWorker.onmessage = async (e) => {
  if (e.data.type === 'config-update') {
    await nginx.suspendAllWorkers();
    applyConfig(e.data.config);
    await nginx.resumeAllWorkers();
  }
};
```

```
// /etc/nginx/js/config_poller.js – JS Worker thread
import * as std from 'std';

let lastETag = '';

while (true) {
  // Poll every 30 seconds
  os.sleep(30000);

  const res = std.urlGet('http://config-service.internal/v1/nginx-config',
    { full: true });
  const etag = (res.headers || {})[ 'etag' ] || '';

  if (res.status === 200 && etag !== lastETag) {
    lastETag = etag;
    postMessage({ type: 'config-update', config: JSON.parse(res.body) });
  }
}
```

14.4. Decision Guide

Requirement	js_preprocess	Pattern C (live)	Full restart
Generate server/location blocks	✓	□ (can addServer/addLocation, but no merge treatment)	✓ (via preprocess)
Network dependency at start time	Risky (prefer file)	✓	Risky
Zero downtime	□ (graceful reload)	✓	□
Upstream peer weights	✓	✓	✓
SSL certificate rotation	✓	✓	✓
New location with full directive inheritance	✓	Partial (addLocation inherits from template only)	✓

15. Third-Party Module Integration

15.1. Overview

Any nginx module can expose its configuration to the COM by implementing a small bridge file that reads the module's C config structs and projects them as JS properties.

This section walks through adding COM support to a hypothetical `ngx_http_mymod` module with a main-level config struct.

15.2. Step 1 — Identify the Config Structs

```
/* ngx_http_mymod.h */

typedef struct {
    ngx_flag_t    enable;
    ngx_uint_t    max_entries;
    ngx_str_t     backend_url;
} ngx_http_mymod_main_conf_t;

typedef struct {
    ngx_msec_t    timeout_ms;
    ngx_flag_t    cache;
} ngx_http_mymod_loc_conf_t;

extern ngx_module_t  ngx_http_mymod_module;
```

15.3. Step 2 — Write the COM Bridge File

Create `ngx_js_com_mymod.c` next to your module sources:

```
/* ngx_js_com_mymod.c */
#include <ngx_config.h>
#include <ngx_core.h>
#include <ngx_http.h>
#include "quickjs.h"
#include "ngx_js.h"
#include "ngx_http_mymod.h"

/*
 * Getter / setter for main-conf properties.
 * 'magic' selects which property (same pattern as NginxRequest).
 */
static JSValue
ngx_js_mymod_main_get(JSContext *ctx, JSValueConst this_val, int magic)
{
```

```

ngx_http_mymod_main_conf_t  *mcf;

/* Retrieve the main config from the cycle. */
mcf = ngx_http_cycle_get_module_main_conf(ngx_cycle,
                                           ngx_http_mymod_module);

if (mcf == NULL) {
    return JS_UNDEFINED;
}

switch (magic) {
case 0: return JS_NewBool(ctx, mcf->enable);
case 1: return JS_NewUint32(ctx, mcf->max_entries);
case 2: return JS_NewStringLen(ctx, (char *) mcf->backend_url.data,
                               mcf->backend_url.len);
}
return JS_UNDEFINED;
}

static JSValue
ngx_js_mymod_main_set(JSContext *ctx, JSValueConst this_val,
                     JSValueConst val, int magic)
{
    ngx_http_mymod_main_conf_t  *mcf;

    mcf = ngx_http_cycle_get_module_main_conf(ngx_cycle,
                                              ngx_http_mymod_module);

    if (mcf == NULL) {
        return JS_EXCEPTION;
    }

    switch (magic) {
case 0:
    mcf->enable = JS_ToBool(ctx, val);
    break;
case 1:
    JS_ToUint32(ctx, &mcf->max_entries, val);
    break;
/* backend_url is an ngx_str_t backed by pool memory – omit setter
   or implement with careful pool allocation */
}
    return JS_UNDEFINED;
}

/*
 * Install nginx.http.mymod into ctx.
 * Called from ngx_js_install_nginx_com() during init_conf.
 */
ngx_int_t
ngx_js_com_mymod_install(JSContext *ctx, JSValueConst nginx_obj)
{
    JSValue  http_obj, mymod_obj;

```

```

/* Retrieve the existing nginx.http object. */
http_obj = JS_GetPropertyStr(ctx, ngx_obj, "http");
if (JS_IsUndefined(http_obj)) {
    return NGX_ERROR;
}

mymod_obj = JS_NewObject(ctx);
if (JS_IsException(mymod_obj)) {
    JS_FreeValue(ctx, http_obj);
    return NGX_ERROR;
}

/* Expose enable / maxEntries / backendUrl as JS properties. */
JS_DefinePropertyGetSet(ctx, mymod_obj,
    JS_NewAtom(ctx, "enable"),
    JS_NewCFunctionMagic(ctx, ngx_js_mymod_main_get, "get_enable", 0,
        JS_CFUNC_generic_magic, 0),
    JS_NewCFunctionMagic(ctx, ngx_js_mymod_main_set, "set_enable", 1,
        JS_CFUNC_generic_magic, 0),
    JS_PROP_CONFIGURABLE);

JS_DefinePropertyGetSet(ctx, mymod_obj,
    JS_NewAtom(ctx, "maxEntries"),
    JS_NewCFunctionMagic(ctx, ngx_js_mymod_main_get, "get_maxEntries", 0,
        JS_CFUNC_generic_magic, 1),
    JS_NewCFunctionMagic(ctx, ngx_js_mymod_main_set, "set_maxEntries", 1,
        JS_CFUNC_generic_magic, 1),
    JS_PROP_CONFIGURABLE);

JS_DefinePropertyGetSet(ctx, mymod_obj,
    JS_NewAtom(ctx, "backendUrl"),
    JS_NewCFunctionMagic(ctx, ngx_js_mymod_main_get, "get_backendUrl", 0,
        JS_CFUNC_generic_magic, 2),
    JS_UNDEFINED, /* no setter – read-only */
    JS_PROP_CONFIGURABLE);

JS_SetPropertyStr(ctx, http_obj, "mymod", mymod_obj);
JS_FreeValue(ctx, http_obj);

return NGX_OK;
}

```

15.4. Step 3 — Register the Bridge in the COM Initialiser

In `ngx_js_com.c`, locate `ngx_js_install_nginx_com` and add a call to your bridge function, guarded by whether the module was compiled in:

```

/* ngx_js_com.c - in ngx_js_install_nginx_com() */
#if defined(NGX_HTTP_MYMOD)
    if (ngx_js_com_mymod_install(ctx, nginx_obj) != NGX_OK) {
        return NGX_ERROR;
    }
#endif

```

15.5. Step 4 — Use from JavaScript

```

// init.js
nginx.log(nginx.INFO, 'mymod enabled: ${nginx.http.mymod.enable}');
nginx.log(nginx.INFO, 'mymod maxEntries: ${nginx.http.mymod.maxEntries}');

// Toggle mymod from a handler (takes effect immediately for this worker)
nginx.http.servers[0].locations
    .find(l => l.path === '/admin/mymod-toggle')
    .handler = (req) => {
        nginx.http.mymod.enable = !nginx.http.mymod.enable;
        req.text(`mymod is now ${nginx.http.mymod.enable ? 'on' : 'off'}\n`);
    };

```

15.6. Location-Level Config

For per-location config, retrieve the conf pointer from the request's `loc_conf` array rather than the cycle:

```

/* In a JS getter called from a request handler context */
static JSValue
ngx_js_mymod_loc_get_timeout(JSContext *ctx, JSValueConst this_val, int magic)
{
    ngx_http_request_t      *r;
    ngx_js_worker_t          *w;
    ngx_http_mymod_loc_conf_t *lcf;

    w = JS_GetContextOpaque(ctx);
    r = w->current_request;
    if (r == NULL) {
        return JS_UNDEFINED;
    }

    lcf = ngx_http_get_module_loc_conf(r, ngx_http_mymod_module);
    return JS_NewUint32(ctx, lcf->timeout_ms);
}

```

16. Resource Protection

16.1. Worker Memory Limit

`nginx.workerMemoryLimit` caps the QuickJS heap for all JS allocations in a worker process. The limit applies on top of nginx's own memory — it measures only memory allocated through the QuickJS allocator.

```
// Set from js_source – applies to every worker after fork().
nginx.workerMemoryLimit = 128 * 1024 * 1024; // 128 MB per worker

// Read back the configured limit (0 = unlimited).
nginx.log(nginx.INFO, `Memory limit: ${nginx.workerMemoryLimit}`);
```

When a JS allocation exceeds the limit, QuickJS throws `RangeError: out of memory`. The worker process remains healthy; the individual request receives a 500 error.

```
loc.handler = (req) => {
  try {
    const huge = new ArrayBuffer(512 * 1024 * 1024); // exceeds limit
    req.text('allocated\n');
  } catch (e) {
    // e is RangeError: out of memory
    req.respond(503, {}, 'memory limit exceeded, try again\n');
  }
};
```

16.2. Worker Request Timeout

`nginx.workerRequestTimeout` sets the maximum wall-clock time (in milliseconds) a single JS handler invocation may consume. If the handler runs longer than this (including time spent in a tight synchronous loop), QuickJS fires an interrupt that aborts the handler. The worker survives and the request receives HTTP 500.

```
// Allow each handler at most 5 seconds of wall time.
nginx.workerRequestTimeout = 5000;

// Re-configure at runtime from a handler.
nginx.http.servers[0].locations
  .find(l => l.path === '/admin/timeout')
  .handler = (req) => {
    const ms = parseInt(req.queryParams.ms);
    if (ms >= 0) nginx.workerRequestTimeout = ms;
    req.text(`timeout is now ${nginx.workerRequestTimeout} ms\n`);
  };
```

16.3. Error Isolation

JavaScript exceptions in a handler are caught by the module framework and converted to HTTP 500 responses. The worker process never crashes due to an unhandled JS exception.

```
loc.handler = (req) => {  
  // This exception is caught by the framework – worker stays up.  
  throw new Error('deliberate error');  
  // Client receives: HTTP/1.1 500 Internal Server Error  
};
```

16.4. What JS Cannot Do

The JS runtime runs inside an nginx worker process (or master, for SharedWorkers) with the same OS privileges. The following restrictions apply:

- No access to nginx's internal C memory outside the COM API. There is no `ctypes`, `ffi`, or equivalent.
- No subprocess spawning from handlers or SharedWorkers. The `os.exec()` function from QuickJS `os` is available only in Workers and `js_preprocess` scripts, not in nginx event-loop handlers.
- Blocking the event loop with a synchronous tight loop is interrupted by the request timeout if `nginx.workerRequestTimeout` is set.
- `Atomics.wait` (blocking) must not be called from the event loop; use `Atomics.waitAsync` (non-blocking, returns a Promise) instead.

17. Reference

17.1. Directives

Directive	Context	Argument s	Description
<code>js_source path</code>	main	1 (path)	Evaluate <code>path</code> in the master process after config parsing (<code>init_conf</code>). Multiple allowed; evaluated in declaration order. The JS runtime created here is inherited by all workers after fork.
<code>js_preprocess path</code>	main	1 (path)	Evaluate <code>path</code> immediately when encountered during <code>ngx_conf_parse</code> . Receives global <code>config.write(text)</code> . Multiple allowed; each runs in an isolated ephemeral runtime.
<code>js_init_http path</code>	http	1 (path)	Evaluate <code>path</code> after the <code>http {}</code> block finishes parsing, before merge. Receives <code>config.write(text)</code> and read-only <code>nginx.http.servers[]</code> .

17.2. **nginx** Global Object

Property / Method	Type	Description
<code>nginx.version</code>	string	nginx version string ("1.27.x")
<code>nginx.cpuCount</code>	number	Number of CPU cores
<code>nginx.workerMemoryLimit</code>	number	QuickJS heap cap in bytes; 0 = unlimited (r/w)
<code>nginx.workerRequestTimeout</code>	number	Max handler wall time in ms; 0 = unlimited (r/w)
<code>nginx.log(level, msg)</code>	—	Write to nginx error log
<code>nginx.DEBUG/INFO/NOTICE/WARN/ERR/CRIT/ALERT/EMERG</code>	number	Log level constants
<code>nginx.broadcast(msg)</code>	—	Deliver <code>msg</code> to <code>nginx.onbroadcast</code> on every worker
<code>nginx.onbroadcast</code>	function	Set handler for broadcast messages (r/w)
<code>nginx.setTimeout(ms)</code>	Promise	Resolve after <code>ms</code> ms (non-blocking)
<code>nginx.cycle</code>	object	Cycle-level stats (connections, openFilesN, sockets)
<code>nginx.events</code>	object	<code>events{}</code> directives (<code>workerConnections</code> , <code>multiAccept</code> , ...)
<code>nginx.http</code>	object	HTTP COM tree
<code>nginx.stream</code>	object	Stream COM tree
<code>nginx.suspendAcceptance()</code>	—	Remove listening sockets from this worker's epoll
<code>nginx.resumeAcceptance()</code>	—	Re-add listening sockets to this worker's epoll
<code>nginx.suspendAllWorkers()</code>	Promise	Quiesce all workers (async)
<code>nginx.resumeAllWorkers()</code>	Promise	Resume all workers (async)

17.3. **nginx.http** Object

Property / Method	Type	Description
<code>servers[]</code>	array	Array of <code>NginxServer</code> objects
<code>upstreams[]</code>	array	Array of <code>NginxUpstream</code> objects
<code>variables</code>	object	nginx HTTP variables (r/w)
<code>serverNamesHashMaxSize</code>	number	r/w
<code>serverNamesHashBucketSize</code>	number	r/w
<code>variablesHashMaxSize</code>	number	r/w
<code>variablesHashBucketSize</code>	number	r/w
<code>sockets[]</code>	array	Listening socket objects
<code>addServer(name)</code>	<code>NginxServer</code>	Create a new virtual server at runtime
<code>removeServer(name)</code>	boolean	Remove a virtual server; returns false if not found

Property / Method	Type	Description
<code>rebuildVhostDispatch()</code>	—	Recompile the <code>server_name</code> dispatch hash
<code>match(req, conditions)</code>	boolean	Evaluate nginx <code>if</code> conditions against a request

17.4. NginxRequest Methods and Properties

See [\[nginxrequest-reference\]](#).

17.5. Worker / SharedWorker API

JS DOM implements a subset of the standard Web Worker and SharedWorker APIs.

API	Notes
<code>new Worker(url)</code>	Spawn a JS Worker thread. <code>url</code> is a path to a JS file.
<code>worker.postMessage(data [, transferList])</code>	Send a value to the Worker. Transferable: <code>ArrayBuffer</code> . Shareable: <code>SharedArrayBuffer</code> .
<code>worker.onmessage = fn</code>	Receive a value from the Worker.
<code>worker.terminate()</code>	Stop the Worker thread.
<code>new SharedWorker(url)</code>	Connect to (or create) a SharedWorker. Returns an object with a <code>port</code> property.
<code>sw.port.postMessage(data)</code>	Send to the SharedWorker.
<code>sw.port.onmessage = fn</code>	Receive from the SharedWorker.
<code>onconnect = fn</code> (in SW script)	Fires when a new worker process connects. <code>e.ports[0]</code> is the port.
<code>onmessage = fn</code> (in SW script)	Global handler; fires for any message from any worker. <code>clients</code> array is available.
<code>clients</code> (in SW script)	Array of all connected port objects; use for broadcast.

17.6. SharedArrayBuffer and Atomics Support Matrix

Operation	Event loop	JS Worker	SharedWorker	<code>js_preprocess</code>
<code>new SharedArrayBuffer(n)</code>	✓	✓	✓	✓
Typed array read/write	✓	✓	✓	✓
<code>Atomics.load/store/add/...</code>	✓	✓	✓	✓
<code>Atomics.compareExchange</code>	✓	✓	✓	✓
<code>Atomics.wait</code> (blocking)	□	✓	✓	✓
<code>Atomics.waitAsync</code>	✓	✓	✓	□

Operation	Event loop	JS Worker	SharedWorker	js_preprocess
<code>Atoms.notify</code>	✓	✓	✓	✓
Transfer via <code>postMessage</code>	✓	✓	✓	—



Never call `Atoms.wait` from the nginx event loop — it blocks the thread, preventing all request processing. Use `Atoms.waitAsync` in the event loop and `Atoms.wait` in Workers / SharedWorker threads.

17.7. Log Level Constants

Constant	nginx log level
<code>nginx.DEBUG</code>	<code>debug</code>
<code>nginx.INFO</code>	<code>info</code>
<code>nginx.NOTICE</code>	<code>notice</code>
<code>nginx.WARN</code>	<code>warn</code>
<code>nginx.ERR</code>	<code>error</code>
<code>nginx.CRIT</code>	<code>crit</code>
<code>nginx.ALERT</code>	<code>alert</code>
<code>nginx.EMERG</code>	<code>emerg</code>

18. Appendix — Flow Charts

18.1. A: Simple Synchronous Handler

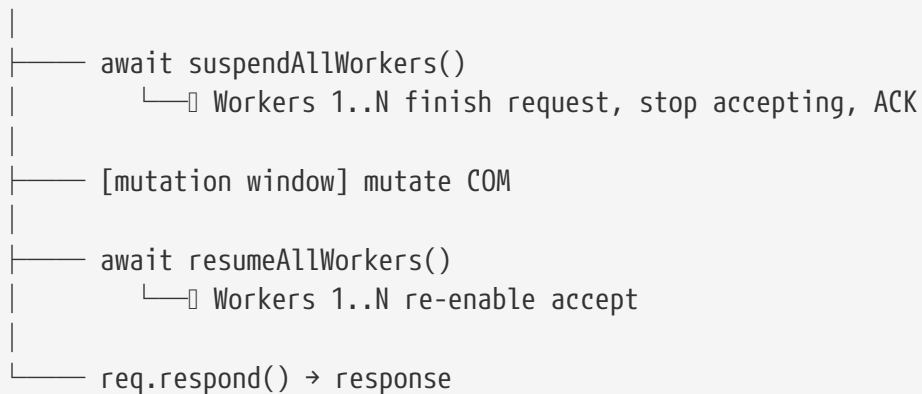
Client → nginx → location matched → handler(req) → req.json() → response

18.2. B: Async Handler with Worker Offload

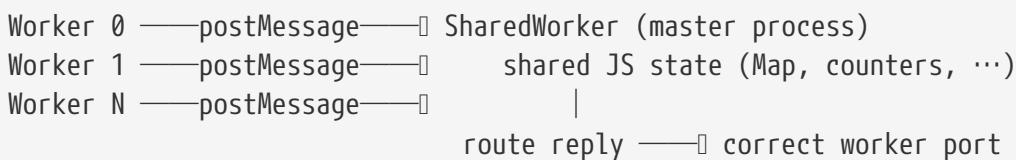


18.3. C: Cross-Worker COM Mutation (suspendAllWorkers)

Client → handler



18.4. D: SharedWorker State Coordination



18.5. E: SAB Lock-Free Counter (Pre-Fork)

```
js_source: const sab = new SharedArrayBuffer(8); ← master creates
fork() x N workers                               ← each inherits CoW pages
Worker A: Atomics.add(hits, 0, 1n) ——— kernel mediates CAS on shared page
Worker B: Atomics.add(hits, 0, 1n) ——— (no user-space lock needed)
```

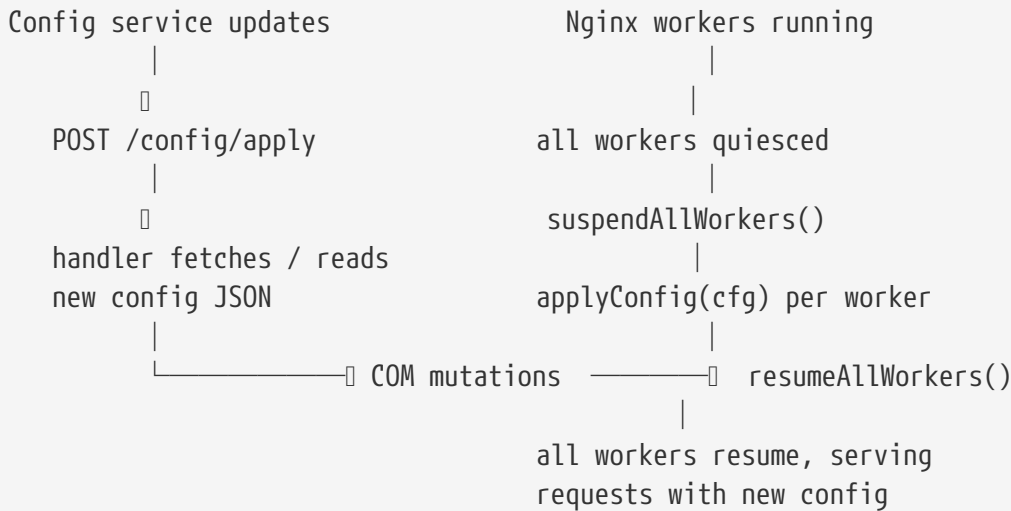
The diagram illustrates a lock-free counter using a SharedArrayBuffer (SAB). The master process creates a SAB of size 8. It then forks N workers. Each worker inherits a copy-on-write (CoW) page of the SAB. Worker A and Worker B both call `Atomics.add(hits, 0, 1n)`. The kernel mediates the CAS (Compare-and-Swap) operation on the shared page, ensuring that no user-space lock is needed.

18.6. F: Pre-Fork Config Generation

```
nginx start → conf parse → js_preprocess fires
|
| JS reads files / env / network
|
| config.write(generated_text)
|
| text re-enters parser inline
|
| server{}/location{} blocks created with
full merge/init_locations/optimize_servers
```

The diagram shows the flow of configuration generation in Nginx. It starts with `nginx start`, followed by `conf parse`, which then triggers `js_preprocess` to fire. The JavaScript code then reads files, environment variables, and network settings. It writes the generated configuration text to `config.write(generated_text)`. This text then re-enters the parser inline, and finally, the `server{}/location{}` blocks are created with full merge/init_locations/optimize_servers.

18.7. G: Live Reload from Config Service



18.8. H: Complete nginx.* Object Tree

The following tree lists every property and method reachable from the global `nginx` object, with types and mutability. `r/o` = read-only; `r/w` = read-write; `fn` = function / method.

```
nginx (global object)
├── version                string  r/o    nginx version string, e.g. "1.27.4"
├── workerMemoryLimit      number  r/w    QuickJS heap cap per worker, bytes (0 =
unlimited)
├── workerRequestTimeout   number  r/w    Per-handler wall-time cap, ms (0 =
unlimited)
├── log(level, msg)         fn      Write to nginx error log
├── broadcast(msg)         fn      Deliver msg to nginx.onbroadcast on every
worker
├── onbroadcast             fn      r/w    Handler called when a broadcast
arrives
├── setTimeout(ms)         Promise  fn      Resolve after ms milliseconds (non-
blocking)
├── suspendAcceptance()    fn      Remove listening sockets from this
worker's epoll
├── resumeAcceptance()     fn      Re-add listening sockets to this worker's
epoll
├── suspendAllWorkers()    Promise  fn      Quiesce all workers (async)
├── resumeAllWorkers()     Promise  fn      Resume all workers (async)
├── cycle (NginxCycle)
│   ├── hostname           string  r/o    system hostname
│   ├── prefix             string  r/o    conf directory (dirname of nginx.conf)
│   ├── confFile           string  r/o    full path to nginx.conf
│   └── errorLog           string  r/o    path to error log file
```

		installPrefix	string	r/o	nginx --prefix
		pid	string	r/o	path to PID file
		connectionN	number	r/o	worker_connections value
		workers	number	r/w	worker_processes count
		daemon	bool	r/o	daemon on off
		master	bool	r/o	master_process on off
		timerResolution	number	r/o	timer_resolution in ms (0 = disabled)
		shutdownTimeout	number	r/o	worker_shutdown_timeout in ms (0 = disabled)
		priority	number	r/o	worker_priority (nice value)
		rlimitNofile	number	r/o	worker_rlimit_nofile (-1 if unset)
		workingDirectory	string	r/o	working_directory (" " if unset)
		sockets[]	array	r/o	NginxSocket – listening sockets snapshot
		events (NginxEvents)			
		connections	number	r/o	worker_connections
		use	string	r/o	event method ("epoll", "kqueue", ...)
		multiAccept	bool	r/w	multi_accept on off
		acceptMutex	bool	r/w	accept_mutex on off
		acceptMutexDelay	number	r/w	accept_mutex_delay in ms
		http (object)			
		servers[]	array	r/o	NginxServer
		upstreams[]	array	r/o	NginxUpstream
		variables	object	r/o	variable name → flags/type map
		serverNamesHashMaxSize	number	r/o	
		serverNamesHashBucketSize	number	r/o	
		variablesHashMaxSize	number	r/o	
		variablesHashBucketSize	number	r/o	
		sockets[]	array	r/o	NginxSocket – HTTP listening sockets
		addServer(name)	fn		→ NginxServer create virtual server
		at runtime			
		removeServer(name)	fn		→ bool remove virtual server
		rebuildVhostDispatch()	fn		recompile server_name dispatch hash
		(call after addServer/removeServer/setNames)			
		match(req, conds)	fn		→ bool evaluate nginx if-conditions
		stream (object)			
		servers[]	array	r/o	NginxStreamServer
		upstreams[]	array	r/o	NginxStreamUpstream

18.8.1. NginxServer properties

NginxServer (nginx.http.servers[i])				
—	name	string	r/o	primary server_name
—	names[]	array	r/o	all server_name values
—	listen[]	array	r/o	listen addresses

locations[]	array	r/o	NginxLocation
clientHeaderBufferSize	number	r/w	bytes
clientHeaderTimeout	number	r/w	ms
connectionPoolSize	number	r/w	bytes
requestPoolSize	number	r/w	bytes
ignoreInvalidHeaders	bool	r/w	
mergeSlashes	bool	r/w	
underscoresInHeaders	bool	r/w	
serverTokens	bool	r/w	
ssl			NginxSSL (see below)
setNames(names[])		fn	
addLocation(path [, {template}])	fn		→ NginxLocation
removeLocation(path)	fn		→ bool
findLocation(path)	fn		→ NginxLocation null
clone(name)	fn		→ NginxServer

18.8.2. NginxLocation properties

NginxLocation (server.locations[i] or server.addLocation(...))			
path	string	r/o	location path
pattern	string	r/o	compiled regex (regex locations only)
matchType	string	r/o	"prefix" "exact" "regex" "named_regex"
internal	bool	r/o	
jsCreated	bool	r/o	true if created by addLocation()
handler		r/w	JS async (req) => {} content handler
headerFilters[]	array	r/o	installed header filter entries
bodyFilters[]	array	r/o	installed body filter entries
root	string	r/w	
alias	string	r/w	
index	string	r/w	
defaultType	string	r/w	
sendfile	bool	r/w	
tcpNopush	bool	r/w	
tcpNodelay	bool	r/w	
etag	bool	r/w	
readAhead	number	r/w	bytes
directio	number	r/w	bytes (0 = disabled)
directioAlignment	number	r/w	bytes
satisfy	string	r/w	"all" "any"
limitExcept	string	r/w	
lingering	bool	r/w	
lingeringTimeout	number	r/w	ms
lingeringTime	number	r/w	ms

```

|—— keepaliveTimeout  number  r/w  ms
|—— keepaliveRequests number  r/w
|—— keepaliveTime     number  r/w  ms
|—— keepaliveDisable  string  r/w  "none" | "msie6" | ...
|—— keepaliveMinTimeout number r/w  ms
|—— clientMaxBodySize  number  r/w  bytes
|—— clientBodyTimeout  number  r/w  ms
|—— clientBodyBufferSize number r/w  bytes
|—— clientBodyInFileOnly bool    r/w
|—— clientBodyInSingleBuffer bool   r/w
|—— sendTimeout        number  r/w  ms
|—— sendLowat          number  r/w  bytes
|—— postponeOutput     number  r/w  bytes
|—— sendfileMaxChunk   number  r/w  bytes
|—— resetTimedoutConnection bool   r/w
|—— absoluteRedirect   bool     r/w
|—— serverNameInRedirect bool     r/w
|—— portInRedirect     bool     r/w
|—— chunkedTransferEncoding bool   r/w
|—— resolverTimeout    number  r/w  ms
|—— authDelay          number  r/w  ms
|—— maxRanges          number  r/w
|—— ifModifiedSince    string  r/w  "off" | "exact" | "before"
|—— msieRefresh        bool     r/w
|—— msiePadding        bool     r/w
|—— logNotFound        bool     r/w
|—— logSubrequest      bool     r/w
|—— recursiveErrorPages bool     r/w
|—— typesHashMaxSize   number  r/o
|
|—— ——— Module sub-objects (all r/o access; properties within are r/w) ———
|—— proxy              NginxProxy
|—— gzip               NginxGzip
|—— ssl                NginxSSL
|—— headers            NginxHeaders
|—— rewrite            NginxRewrite
|—— access             NginxAccess
|—— auth               NginxAuth
|—— authRequest        NginxAuthRequest
|—— autoindex          NginxAutoindex
|—— charset            NginxCharset
|—— dav                NginxDav
|—— fastcgi            NginxFastCGI
|—— gunzip             NginxGunzip
|—— gzipStatic         NginxGzipStatic
|—— imageFilter        NginxImageFilter
|—— limitConn          NginxLimitConn
|—— limitReq           NginxLimitReq
|—— log                NginxLog
|—— memcached          NginxMemcached
|—— mirror             NginxMirror

```

— mp4	NginxMp4
— proxyCache	NginxProxyCache
— randomIndex	NginxRandomIndex
— realip	NginxRealIP
— referer	NginxReferer
— scgi	NginxScgi
— secureLink	NginxSecureLink
— slice	NginxSlice
— ssi	NginxSsi
— subFilter	NginxSubFilter
— tryFiles	(object)
— userid	NginxUserid
— uwsgi	NginxUwsgi
— addition	NginxAddition
— xslt	NginxXslt
— setWriteMode("global" "local" "both")	fn
— setReadMode("global" "local")	fn
— setProperty(name, value [, mode])	fn
— getProperty(name [, mode])	fn
— addHeaderFilter(name, fn [, priority])	fn
— addBodyFilter(name, fn [, priority])	fn
— removeHeaderFilter(name)	fn
— removeBodyFilter(name)	fn
— getHeaderFilter(name) / getBodyFilter(name)	fn

18.8.3. NginxUpstream, NginxPeer, NginxRRPeer

NginxUpstream (nginx.http.upstreams[i] or nginx.stream.upstreams[i])

— name	string	r/o	
— zone	string	r/o	shared memory zone name (" " if none)
— peers[]	array	r/o	NginxPeer (config time) or NginxRRPeer (runtime)
— addPeer(cfg)	fn		→ NginxPeer NginxRRPeer
— removePeer(addr)	fn		→ bool

NginxPeer (config-time upstream peer)

— address	string	r/o	
— weight	number	r/w	
— maxFails	number	r/w	
— failTimeout	number	r/w	seconds
— maxConns	number	r/w	
— down	bool	r/w	
— backup	bool	r/o	

NginxRRPeer (runtime round-robin peer – same as NginxPeer plus:)

— conns	number	r/o	current active connections
— fails	number	r/o	current fail counter
— server	string	r/o	upstream group name

18.8.4. NginxSocket and NginxHttpListener / NginxStreamListener

```
NginxSocket (nginx.cycle.sockets[i] or nginx.http.sockets[i])
├── address      string r/o  "host:port"
├── port         number r/o
├── fd           number r/o  OS file descriptor
├── listener     r/o        NginxHttpListener | NginxStreamListener | null
├── close()      fn        close and remove socket
└── broadcast()  fn        send fd to all workers via SCM_RIGHTS
```

NginxHttpListener (returned by socket listener attach /
nginx.http.sockets[i].listener)

```
├── address      string r/o
├── socket       NginxSocket r/o
├── serverNames  object r/o  { name → NginxServer }
├── serverByName(name)  fn → NginxServer | null
├── addServer(cfg)      fn → NginxServer
└── addVirtualServer(cfg) fn → NginxServer
```

NginxStreamListener – same shape as NginxHttpListener but returns NginxStreamServer

18.8.5. NginxStreamServer and NginxStreamProxy

```
NginxStreamServer (nginx.stream.servers[i])
├── serverName      string r/o
├── tcpNodelay       bool   r/w
├── prereadBufferSize number r/w bytes
├── prereadTimeout   number r/w ms
├── resolverTimeout  number r/w ms
├── proxyProtocolTimeout number r/w ms
├── proxy            NginxStreamProxy
├── access           NginxStreamAccess
├── ssl              NginxStreamSSL
└── handler          r/w    JS (session) => {} content handler
```

```
NginxStreamProxy
├── pass          string r/o  upstream name
├── connectTimeout number r/w ms
├── timeout        number r/w ms (proxy_timeout)
├── nextUpstreamTimeout number r/w ms
├── bufferSize     number r/w bytes
├── requests       number r/o  proxy_requests value
├── responses      number r/o  proxy_responses value
├── nextUpstreamTries number r/w
├── nextUpstream   string r/w
└── proxyProtocol  bool   r/w
```

halfClose	bool	r/w
socketKeepalive	bool	r/w

18.8.6. NginxRequest and NginxStreamSession

NginxRequest (handler argument in HTTP content handlers)

—— Read-only request info ——

method	string	"GET" "POST" ...
uri	string	path without query string
args	string	raw query string
queryParams	object	parsed {key: value} query parameters
scheme	string	"http" "https"
host	string	Host header value
remoteAddr	string	client IP
remotePort	number	client TCP port
serverAddr	string	server-side IP
serverPort	number	server-side port
httpVersion	string	"1.0" "1.1" "2.0"
keepalive	bool	connection is keep-alive
isInternal	bool	is subrequest
startTime	number	ms since Unix epoch
requestLength	number	total bytes received (headers + body)
contentType	string	Content-Type header value
contentLength	number	Content-Length value
connection	object	nginx connection info
headers	object	request headers (lowercased keys)
cookies	object	parsed cookies {name: value}
upstream	object	upstream peer info (if proxied)

—— Mutable ——

statusCode	number	r/w	response status code
variables	object	r/w	nginx variables exotic proxy (r.variables.foo)
location	NginxLocation	r/o	matched location (mutable via setWriteMode)

—— Response ——

respond(status, headers, body)	fn	send complete response
json(data)	fn	200 + application/json
text(body)	fn	200 + text/plain
html(body)	fn	200 + text/html
redirect(url)	fn	302 redirect
writeHead(status, headers)	fn	begin streaming response
write(chunk)	fn	append streaming chunk
finish()	fn	end streaming response
sendfile(path)	fn	serve file via sendfile(2)

—— Headers ——

setHeader(name, value)	fn	
getHeader(name)	fn	→ string undefined

```

|_____ removeHeader(name)                fn
|
|_____ — Variables —
|_____ variable(name) / getVar(name)      fn → string | number | undefined
|_____ setVariable(name, v) / setVar      fn
|
|_____ — Body —
|_____ body                string r/o      buffered body (null if not yet read)
|_____ readBody()          Promise fn      read full request body → string
|
|_____ — Async —
|_____ sleep(ms)           Promise fn      non-blocking delay
|_____ fetch(url)          Promise fn      non-blocking upstream HTTP request → Response
|_____ subrequest(uri)     Promise fn      nginx internal subrequest → Response
|
|_____ log(level, msg)      fn

```

NginxStreamSession (handler argument in stream content handlers)

```

|_____ remoteAddress  string r/o
|_____ remotePort     number r/o
|_____ localAddress   string r/o
|_____ localPort      number r/o
|_____ received       number r/o      bytes received so far
|_____ status         number r/o      session status code
|_____ ssl            object r/o      SSL info (null on plain TCP)
|_____ finalize([code]) fn          close session; default code = NGX_STREAM_OK
|_____ variable(name)  fn          → string | undefined

```

18.8.7. Worker and SharedWorker

Worker (new Worker(url))

```

|_____ onmessage      fn      r/w      called when the Worker posts a message
|_____ postMessage(data [, transferList]) fn  send value to the Worker
|_____ terminate()     fn      stop the Worker thread

```

SharedWorker (new SharedWorker(url))

```

|_____ port           MessagePort

```

MessagePort (sw.port)

```

|_____ onmessage      fn      r/w      called when the SharedWorker posts a message
|_____ postMessage(data) fn    send value to the SharedWorker

```

— Inside a SharedWorker script —

```

onconnect                fn r/w  fires when a new nginx worker connects;
e.data.ports[0] = MessagePort
onmessage                fn r/w  global handler; receives messages from any worker
clients[]                array   all connected MessagePort objects (for broadcast)

```

18.8.8. NginxSSL and NginxStreamSSL

NginxSSL (server.ssl in http context)

sessionTimeout	number	r/w	seconds
sessionTickets	bool	r/w	
preferServerCiphers	bool	r/w	
verifyDepth	number	r/w	
ciphers	string	r/o	
clientCertificate	string	r/o	
trustedCertificate	string	r/o	
ecdhCurve	string	r/o	
dhparam	string	r/o	
setCiphers(list)	fn		change cipher list
setProtocols(list)	fn		e.g. "TLSv1.2 TLSv1.3"
setCertificate(cert, key)	fn		hot-swap certificate and key

NginxStreamSSL (streamServer.ssl in stream context) – same properties plus:

handshakeTimeout	number	r/w	ms
------------------	--------	-----	----

18.9. I: Full Nginx JS Class Reference

18.9.1. Class Index

Class name	JS constructor / access path	Purpose
<code>NginxCycle</code>	<code>nginx.cycle</code>	Cycle-level nginx globals
<code>NginxEvents</code>	<code>nginx.events</code>	<code>events{}</code> block settings
<code>NginxServer</code>	<code>nginx.http.servers[i]</code>	HTTP virtual server
<code>NginxLocation</code>	<code>server.locations[i],</code> <code>addLocation()</code>	HTTP location block
<code>NginxUpstream</code>	<code>nginx.http.upstreams[i]</code>	HTTP upstream group
<code>NginxPeer</code>	<code>upstream.peers[i]</code> (config time)	Upstream server (static)
<code>NginxRRPeer</code>	<code>upstream.peers[i]</code> (runtime)	Upstream server (live stats)
<code>NginxRequest</code>	handler argument <code>req</code>	HTTP request/response
<code>NginxSocket</code>	<code>nginx.cycle.sockets[i],</code> <code>http.sockets[i]</code>	Bound listening socket fd

Class name	JS constructor / access path	Purpose
<code>NginxHttpListener</code>	<code>socket.listener</code>	HTTP virtual-host router
<code>NginxStreamServer</code>	<code>nginx.stream.servers[i]</code>	Stream (TCP/UDP) server
<code>NginxStreamProxy</code>	<code>streamServer.proxy</code>	Stream proxy settings
<code>NginxStreamListener</code>	<code>socket.listener (stream)</code>	Stream virtual-host router
<code>NginxStreamSession</code>	<code>handler argument session</code>	TCP/UDP session
<code>NginxStreamUpstream</code>	<code>nginx.stream.upstreams[i]</code>	Stream upstream group
<code>NginxStreamPeer</code>	<code>streamUpstream.peers[i] (config)</code>	Stream upstream server (static)
<code>NginxStreamRRPeer</code>	<code>streamUpstream.peers[i] (runtime)</code>	Stream upstream server (live)
<code>Worker</code>	<code>new Worker(url)</code>	Background JS thread (per nginx-worker)
<code>SharedWorker</code>	<code>new SharedWorker(url)</code>	Shared JS thread (one, in master)
<code>MessagePort</code>	<code>sharedWorker.port</code>	Channel endpoint to SharedWorker
<code>NginxSSL</code>	<code>server.ssl</code>	HTTP SSL/TLS settings
<code>NginxStreamSSL</code>	<code>streamServer.ssl</code>	Stream SSL/TLS settings
<code>NginxProxy</code>	<code>location.proxy</code>	<code>proxy_pass</code> and proxy settings
<code>NginxGzip</code>	<code>location.gzip</code>	<code>gzip</code> module settings
<code>NginxHeaders</code>	<code>location.headers</code>	<code>add_header</code> / <code>expires</code> settings
<code>NginxRewrite</code>	<code>location.rewrite</code>	<code>rewrite</code> module settings
<code>NginxAccess</code>	<code>location.access</code>	<code>allow/deny</code> access rules
<code>NginxAuth</code>	<code>location.auth</code>	<code>auth_basic</code> settings
<code>NginxAuthRequest</code>	<code>location.authRequest</code>	<code>auth_request</code> settings
<code>NginxAutoindex</code>	<code>location.autoindex</code>	Directory listing settings

Class name	JS constructor / access path	Purpose
NginxCharset	location.charset	Charset conversion settings
NginxDav	location.dav	WebDAV settings
NginxFastCGI	location.fastcgi	FastCGI proxy settings
NginxGunzip	location.gunzip	On-the-fly decompression
NginxGzipStatic	location.gzipStatic	Pre-compressed file serving
NginxImageFilter	location.imageFilter	Image transformation settings
NginxLimitConn	location.limitConn	limit_conn settings
NginxLimitReq	location.limitReq	limit_req settings
NginxLog	location.log	Access log settings
NginxMemcached	location.memcached	Memcached proxy settings
NginxMirror	location.mirror	Request mirroring settings
NginxMp4	location.mp4	MP4 pseudo-streaming settings
NginxProxyCache	location.proxyCache	Proxy cache settings
NginxRandomIndex	location.randomIndex	Random index file settings
NginxRealIP	location.realip	Real-IP header settings
NginxReferer	location.referer	Referer validation settings
NginxScgi	location.scgi	SCGI proxy settings
NginxSecureLink	location.secureLink	Secure link settings
NginxSlice	location.slice	Response slicing settings
NginxSsi	location.ssi	SSI processing settings
NginxSubFilter	location.subFilter	Response substitution settings
NginxUserid	location.userid	User-ID cookie settings
NginxUwsgi	location.uwsgi	uWSGI proxy settings
NginxAddition	location.addition	Response addition settings
NginxXslt	location.xslt	XSLT transformation settings

Class name	JS constructor / access path	Purpose
<code>NginxStreamAccess</code>	<code>streamServer.access</code>	Stream access rules

18.9.2. Module COM Class Properties

Class	Property	Mode	Notes
<code>NginxProxy</code>	<code>pass</code>	r/o	upstream URL or name
<code>NginxProxy</code>	<code>httpVersion</code>	r/w	"1.0" or "1.1"
<code>NginxProxy</code>	<code>connectTimeout</code>	r/w	ms
<code>NginxProxy</code>	<code>sendTimeout</code>	r/w	ms
<code>NginxProxy</code>	<code>readTimeout</code>	r/w	ms
<code>NginxProxy</code>	<code>buffering</code>	r/w	bool
<code>NginxProxy</code>	<code>requestBuffering</code>	r/w	bool
<code>NginxProxy</code>	<code>interceptErrors</code>	r/w	bool
<code>NginxProxy</code>	<code>bufferSize</code>	r/w	bytes
<code>NginxProxy</code>	<code>nextUpstreamTries</code>	r/w	number
<code>NginxProxy</code>	<code>nextUpstreamTimeout</code>	r/w	ms
<code>NginxGzip</code>	<code>enable</code>	r/w	bool
<code>NginxGzip</code>	<code>level</code>	r/w	1–9
<code>NginxGzip</code>	<code>minLength</code>	r/w	bytes
<code>NginxGzip</code>	<code>vary</code>	r/w	bool — add Vary: Accept-Encoding
<code>NginxHeaders</code>	<code>expires</code>	r/o	seconds; -1 = epoch; -2 = max; 0 = off
<code>NginxHeaders</code>	<code>expiresTime</code>	r/o	absolute Unix timestamp
<code>NginxHeaders</code>	<code>headersInherit</code>	r/w	bool
<code>NginxHeaders</code>	<code>trailersInherit</code>	r/w	bool
<code>NginxRewrite</code>	<code>log</code>	r/w	bool — log rewrite rule execution
<code>NginxRewrite</code>	<code>uninitializedVariableWarn</code>	r/w	bool
<code>NginxRewrite</code>	<code>stackSize</code>	r/w	number — rewrite stack depth
<code>NginxRewrite</code>	<code>hasRules</code>	r/o	bool — any rules defined
<code>NginxAuth</code>	<code>realm</code>	r/w	string — Basic auth realm
<code>NginxAuth</code>	<code>userFile</code>	r/w	string — path to httpasswd file
<code>NginxAuthRequest</code>	<code>uri</code>	r/w	string — subrequest URI
<code>NginxAutoindex</code>	<code>enable</code>	r/w	bool
<code>NginxAutoindex</code>	<code>format</code>	r/w	"html" "xml" "json" "jsonp"

Class	Property	Mode	Notes
NginxAutoindex	localtime	r/w	bool
NginxAutoindex	exactSize	r/w	bool
NginxCharset	charset	r/o	string
NginxCharset	sourceCharset	r/o	string
NginxCharset	overrideCharset	r/w	bool
NginxCharset	setCharset(from, to)	fn	add charset conversion mapping
NginxDav	methods	r/w	string[] — allowed HTTP methods
NginxDav	access	r/w	number — octal permissions (e.g. <code>0o644</code>)
NginxDav	minDeleteDepth	r/w	number
NginxDav	createFullPutPath	r/w	bool
NginxFastCGI	pass	r/o	string
NginxFastCGI	index	r/w	string
NginxFastCGI	keepConn	r/w	bool
NginxFastCGI	connectTimeout	r/w	ms
NginxFastCGI	sendTimeout	r/w	ms
NginxFastCGI	readTimeout	r/w	ms
NginxFastCGI	buffering	r/w	bool
NginxFastCGI	requestBuffering	r/w	bool
NginxFastCGI	interceptErrors	r/w	bool
NginxGunzip	enable	r/w	bool
NginxGzipStatic	enable	r/w	bool — serve pre-compressed <code>.gz</code> files
NginxImageFilter	action	r/w	<code>"test"</code> <code>"resize"</code> <code>"crop"</code> <code>"rotate"</code>
NginxImageFilter	width	r/w	pixels
NginxImageFilter	height	r/w	pixels
NginxImageFilter	angle	r/w	90 180 270
NginxImageFilter	jpegQuality	r/w	1–100
NginxImageFilter	webpQuality	r/w	1–100
NginxImageFilter	sharpen	r/w	0–100
NginxImageFilter	transparency	r/w	bool
NginxImageFilter	interlace	r/w	bool
NginxImageFilter	bufferSize	r/w	bytes
NginxLimitReq	logLevel	r/w	log level string

Class	Property	Mode	Notes
NginxLimitReq	delayLogLevel	r/w	log level string
NginxLimitReq	statusCode	r/w	HTTP status for rejected requests (default 503)
NginxLimitReq	dryRun	r/w	bool — count but do not limit
NginxLimitConn	logLevel	r/w	log level string
NginxLimitConn	statusCode	r/w	HTTP status (default 503)
NginxLimitConn	dryRun	r/w	bool
NginxLog	off	r/w	bool — disable access logging for this location
NginxMemcached	connectTimeout	r/w	ms
NginxMemcached	sendTimeout	r/w	ms
NginxMemcached	readTimeout	r/w	ms
NginxMemcached	bufferSize	r/w	bytes
NginxMemcached	gzipFlag	r/w	number — Memcached flags value for gzip content
NginxMirror	uris	r/w	string[] — mirror target URIs
NginxMirror	requestBody	r/w	bool — mirror request body
NginxMp4	bufferSize	r/w	bytes
NginxMp4	maxBufferSize	r/w	bytes
NginxMp4	startKeyFrame	r/w	bool
NginxRandomIndex	enable	r/w	bool
NginxRealIP	header	r/o	string — header used as client IP source
NginxRealIP	recursive	r/w	bool
NginxReferer	noReferer	r/w	bool — allow requests without Referer
NginxReferer	blockedReferer	r/w	bool — allow requests with blocked Referer
NginxReferer	serverNames	r/w	string[] — allowed referer server names
NginxScgi	connectTimeout	r/w	ms
NginxScgi	sendTimeout	r/w	ms
NginxScgi	readTimeout	r/w	ms
NginxScgi	bufferSize	r/w	bytes
NginxUwsgi	connectTimeout	r/w	ms

Class	Property	Mode	Notes
NginxUwsgi	sendTimeout	r/w	ms
NginxUwsgi	readTimeout	r/w	ms
NginxUwsgi	bufferSize	r/w	bytes
NginxSecureLink	secret	r/w	string — HMAC secret
NginxSecureLink	variable	r/o	string — variable name holding the link
NginxSecureLink	md5	r/o	bool — MD5-based (vs HMAC-SHA1)
NginxSlice	size	r/w	bytes — slice size for range-splitting
NginxSsi	enable	r/w	bool
NginxSsi	silentErrors	r/w	bool
NginxSsi	lastModified	r/w	bool
NginxSsi	valueLen	r/w	number — max SSI variable value length
NginxSubFilter	once	r/w	bool — replace only first occurrence
NginxSubFilter	lastModified	r/w	bool
NginxUserid	enable	r/w	bool "v1" "v2" "log"
NginxUserid	name	r/w	string — cookie name
NginxUserid	domain	r/w	string — cookie domain
NginxUserid	path	r/w	string — cookie path
NginxUserid	p3p	r/w	string — P3P header value
NginxUserid	expires	r/o	number — seconds
NginxUserid	mark	r/w	string — version mark character
NginxAddition	addBeforeBody	r/w	string — URI to prepend to response body
NginxAddition	addAfterBody	r/w	string — URI to append to response body
NginxProxyCache	minUses	r/w	number
NginxProxyCache	lock	r/w	bool
NginxProxyCache	lockTimeout	r/w	ms
NginxProxyCache	lockAge	r/w	ms
NginxProxyCache	revalidate	r/w	bool
NginxProxyCache	convertHead	r/w	bool
NginxProxyCache	backgroundUpdate	r/w	bool
NginxXslt	lastModified	r/w	bool

Class	Property	Mode	Notes
NginxXslt	sheetsCount	r/o	number — number of loaded stylesheets